HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND ENGINEERING

Faculty of Advanced Education

FINAL COURSE REPORT

COURSE: EMBEDDED SYSTEM DESIGN

RESEARCH, DESIGN AND IMPLEMENTATION OF AN EMBEDDED 3D SCANNING SYSTEM BASED ON THE ESP32-S3 MICROCONTROLLER AND THE SHARP GP2Y0A51SK0F INFRARED DISTANCE SENSOR

Lecturer: M.Sc. Huỳnh Hoàng Hà

COURSE: ESDS437064E

STUDENTS: Huỳnh Anh Tuấn - 24119096

Lê Thành Tài - 24119080

Phan Anh Khoa – 24119048

Ho Chi Minh City, October 2026

LIST OF GROUP MEMBERS

SEMESTER 1, ACADEMIC YEAR 2026 – 2027

Topic: Research, Design and Implementation of an Embedded 3D Scanning System Based on the ESP32-S3 Microcontroller and the Sharp GP2Y0A51SK0F Infrared Distance Sensor

No.	FULL NAME	STUDENT ID	SHARE (%)
1	Huỳnh Anh Tuấn	24119096	100%
2	Lê Thành Tài	24119080	100%
3	Phan Anh Khoa	24119048	100%

Lecturer's comments:

Presentation:

.....

.....

.....

.....

Total score:

.....

.....

.....

Date: ……/……/……

Lecturer

(Signature and full name)

M.Sc. Huỳnh Hoàng Hà

TABLE OF CONTENTS

ABSTRACT.....6
LIST OF ABBREVIATIONS.....7
LIST OF FIGURES.....8
LIST OF TABLES.....10
CHAPTER 1: OVERVIEW.....11
1.1. Rationale.....11
1.2. Objectives.....11
1.3. Subject and scope.....12
1.4. Research methodology.....12
1.5. Scientific and practical significance.....13
CHAPTER 2: THEORETICAL BACKGROUND.....14
2.1. 3D scanning technologies and point clouds.....14
2.2. Cylindrical-coordinate scanning principle.....15
2.2.1. Geometric model.....15
2.2.2. Conversion to Cartesian coordinates.....16
2.2.3. Resolution and scanning volume.....16
2.3. The Sharp GP2Y0A51SK0F infrared distance sensor.....17
2.3.1. Triangulation measuring principle.....17
2.3.2. Specifications and output characteristic.....18
2.3.3. How the beam interacts with different surfaces.....19
2.3.4. Why the TF-Luna was replaced.....21
2.4. ESP32-S3, FreeRTOS and analog acquisition.....22
2.4.1. The ESP32-S3 microcontroller.....22
2.4.2. FreeRTOS tasks on two cores.....22
2.4.3. ADC2 and Wi-Fi coexistence.....23
2.4.4. From voltage to distance.....23
2.5. Stepper motors, the A4988 driver and the lead screw.....24

2.6. Wi-Fi SoftAP and TCP communication....26
CHAPTER 3: SYSTEM DESIGN....27
3.1. System requirements....27
3.2. Overall block diagram....27
3.3. Detailed design of each block....28
3.3.1. Power block....28
3.3.2. Central processing block....28
3.3.3. Sensor block....29
3.3.4. Motor block....29
3.3.5. Communication block....29
3.4. Wiring diagram and pin allocation....29
3.5. Mechanical design and geometric parameters....32
3.6. Embedded software design....34
3.6.1. Configuration parameters....34
3.6.2. Task architecture and code block diagram....35
3.6.3. Motion task and scan loop....37
3.6.4. Point measurement and filtering....38
3.7. ESP32 – PC communication protocol....40
3.8. PC software design....42
3.8.1. The receiver pcrecieve.py....42
3.8.2. The mesh builder xyztostl.py....42
CHAPTER 4: IMPLEMENTATION AND EXPERIMENTS....45
4.1. Hardware implementation....45
4.2. Unit tests of the functional blocks....46
4.2.1. Driver and motor test....46
4.2.2. Sensor repeatability....47
4.2.3. Wi-Fi and transfer test....47
4.3. Calibration....47
4.3.1. Sensor-to-axis distance and beam offset....47

4.3.2. Calibration table....48
4.4. Software verification by simulation....49
4.5. Scanning experiments....51
4.5.1. Effect of the beam offset....51
4.5.2. Real scan of the paper-wrapped cube....52
4.6. Problems encountered and solutions....54
4.7. Evaluation against the objectives....55
CHAPTER 5: CONCLUSION AND FUTURE WORK....56
5.1. Results achieved....56
5.2. Limitations....56
5.3. Future work....57
5.3.1. Hardware and firmware improvements....57
5.3.2. AI-assisted automatic calibration....57
5.3.3. Scanning the top surface....58
5.3.4. Colour output....58
5.3.5. AI point-cloud completion and repair....58
5.4. Conclusion....58
REFERENCES....59

ABSTRACT

This report presents the design, implementation and evaluation of a low-cost embedded 3D scanner that digitises small objects (up to about 60 mm in radius and 150 mm in height) using the cylindrical-coordinate principle. The object stands on a turntable driven by a NEMA17 stepper motor, while a Sharp GP2Y0A51SK0F infrared triangulation sensor, carried by a second NEMA17 on a T8 lead screw, measures the distance to the object surface layer by layer. The Sharp sensor (range 2–15 cm) replaces the TF-Luna LiDAR of the first prototype, whose 0.2 m minimum range and ±6 cm accuracy are unusable at the 30–90 mm working distance.

An ESP32-S3 microcontroller runs two FreeRTOS tasks on its two cores. The motion task on core 1 drives both A4988 drivers in full-step mode, samples the sensor output on GPIO18 (ADC2 channel 7) and converts it to a radius with a measured calibration table. The communication task on core 0 runs a WiFi access point and streams each finished layer as x, y, z text lines to a PC over TCP, using a START/END framing with ACK/NACK confirmation. Scan data is kept only in RAM. On the PC, `pcrecieve.py` stores the points and `xyztostl.py` builds a closed triangle mesh: it corrects the 14.1 mm lateral beam offset, removes grazing-angle points, fills gaps and writes a binary STL file.

The firmware and PC software were tested in 60 software-in-the-loop runs (68 program runs) covering normal scans, Wi-Fi loss, heavy ADC interference and malformed input. In simulation a 60 × 60 mm cube is reconstructed as 60.7–60.9 mm with a surface error of 0.35 mm RMS. On the real machine a 60 × 60 × 65 mm cube wrapped in matt paper is scanned at about 7.9 s per 1 mm layer with no ADC errors, and a square fitted to its layers has a side of 61.3 mm (+1.3 mm) with a mean face deviation of 1.2 mm, which meets the ±1–2 mm accuracy target. The main remaining error source is the surface- and angle-dependent behaviour of the infrared sensor. Future work includes AI-assisted calibration, top-surface scanning and colour capture.

LIST OF ABBREVIATIONS

Term	Meaning
ADC	Analog-to-digital converter. The ESP32-S3 has two 12-bit SAR ADCs (ADC1, ADC2)
ADC2_CH7	Channel 7 of the second ADC, bonded to GPIO18 on the ESP32-S3
ACK / NACK	Acknowledgement / negative acknowledgement returned by the PC after the END line
AP, SoftAP	Access point: the ESP32-S3 creates its own Wi-Fi network (ScannerESP32)
CAD	Computer-aided design
CAL table	Measured calibration table that maps sensor voltage (mV) to distance (mm)
D	Distance from the sensor face to the turntable axis (90 mm)
e (beam offset)	Lateral distance between the infrared beam and the turntable axis (≈14.1 mm)
EN / STEP / DIR	Enable, step-pulse and direction inputs of the A4988 driver
FreeRTOS	Real-time operating system used by ESP-IDF and Arduino-ESP32
GPIO	General-purpose input/output pin
IR	Infrared
LiDAR	Light detection and ranging
PCB	Printed circuit board
PSD	Position-sensitive detector inside the Sharp sensor
RMS	Root mean square
SAR	Successive-approximation-register (ADC architecture)
STL	STereoLithography mesh file format (binary variant used here)
TCP	Transmission Control Protocol
ToF	Time of flight
Vo	Analog output voltage of the Sharp sensor

LIST OF FIGURES

Figure 2.1. Classification of 3D scanning methods; the highlighted block is the method used in this project.....14

Figure 2.2. Geometric model of the scanner: (a) top view of one layer with the beam offset e, (b) side view of the layer-by-layer scan.....16

Figure 2.3. Measuring principle of the Sharp GP2Y0A51SK0F: the reflected spot falls on a different position of the PSD for a near and a far object.....17

Figure 2.4. (a) Measured output voltage of the sensor used in this project, on a flat white board and on the paper-wrapped cube; (b) sensitivity and the distance represented by one ADC step.....19

Figure 2.5. How the infrared beam is reflected by different surfaces and what the receiver sees.....20

Figure 2.6. Measuring distance through the ADC: (a) signal chain, (b) timing of one point, (c) trimmed mean, (d) interpolation in the calibration table.....24

Figure 2.7. Illustrative turntable ringing after one 3.6° move: two steps half a period apart cancel most of the oscillation before the measurement window.....25

Figure 3.1. Overall block diagram of the 3D scanning system.....28

Figure 3.2. Wiring between the ESP32-S3, the two A4988 drivers, the Sharp sensor and the power supply.....30

Figure 3.3. Circuit schematic of the control board (the TF-Luna headers are reused for the Sharp sensor).....31

Figure 3.4. PCB layout of the control board.....32

Figure 3.5. CAD model of the 3D-printed frame: (a) isometric, (b) front, (c) side and (d) top views.....33

Figure 3.6. Block diagram of the main scan firmware: two FreeRTOS tasks connected only through shared RAM.....36

Figure 3.7. Flowchart of the point measurement function irMeasure().....39

Figure 3.8. Sequence diagram of the data exchange between the ESP32-S3 and the PC41

Figure 3.9. Processing pipeline of xyztostl.py.....42

Figure 4.1. Hardware build: (a) printed frame, (b) component side and (c) copper side of the etched control board.....45
Figure 4.2. The assembled scanner: (a) top view, (b) sensor carriage on the Z axis.…46
Figure 4.3. Reference test object and sensor position.....46
Figure 4.4. Calibration-table simulation: (a) beam offset recovered by the auto search, (b) cube size in the final STL with the current and the smoothed table.....48
Figure 4.5. Effect of measurement settings on random error and scan time (simulation, 60 mm cube).....51
Figure 4.6. Effect of the 14.1 mm beam offset on a simulated 60 mm cube: (a) one raw layer, (b) STL without correction, (c) STL with correction.....51
Figure 4.7. Layer statistics from the Serial log of a real scan of the paper-wrapped cube (earlier run with 0.6 mm layers).....52
Figure 4.8. STL models: (a) reference, (b) simulated scan, (c) first real scan with masking tape, (d) final real scan of the paper-wrapped cube.....53

LIST OF TABLES

Table 2.1. Qualitative comparison of 3D scanning methods (compiled from [3], [7], [8]) 15

Table 2.2. Main specifications of the Sharp GP2Y0A51SK0F [4], [11] 18

Table 2.3. Comparison between the TF-Luna and the Sharp GP2Y0A51SK0F for this scanner [8], [4] 21

Table 2.4. ESP32-S3 features used in this project [5], [14], [15] 22

Table 3.1. Functional and non-functional requirements 27

Table 3.2. ESP32-S3 pin allocation (as defined in scan.cpp) 30

Table 3.3. Geometric parameters of the scanning mechanism 33

Table 3.4. Main configuration parameters of the firmware 34

Table 3.5. Function blocks of scan.cpp 37

Table 3.6. Messages of the scan data protocol (one ASCII line each, ending with \n) .41

Table 3.7. Command-line options of xyztostl.py 44

Table 4.1. Repeated readings on a flat white board (each reading is already an average of 16 samples) 47

Table 4.2. Software-in-the-loop test results 49

Table 4.3. Dimensional results for the 60 × 60 × 65 mm cube 53

Table 4.4. Main problems met during development and how they were solved 54

CHAPTER 1: OVERVIEW

1.1. Rationale

3D scanning turns a physical object into a digital model that can be measured, modified in CAD software, or printed again. It is used in reverse engineering, quality inspection, cultural-heritage archiving and education. Commercial scanners based on structured light or laser lines give sub-millimetre accuracy but cost from several hundred to several thousand US dollars, which is out of reach for most student laboratories. Open projects such as FabScan showed that a turntable, a camera and a line laser can produce useful models for about 100 EUR [1].

ElectroNoobs published a closely related Arduino project that uses the same Sharp GP2Y0A51SK0F, two NEMA17 motors with A4988 drivers and an 8 mm lead screw [2]. It averages 40 ADC readings per point, converts voltage to distance with a cubic polynomial, logs the points to an SD card and meshes them later in MeshLab; its author notes that the precision is not very good. The present project keeps the same low-cost idea but moves the processing to a dual-core ESP32-S3, adds wireless streaming, robust sampling and a measured calibration table, and corrects the sensor mounting geometry in software.

A scanner built around a single-point distance sensor is even simpler: it needs no camera, no image processing and no optical calibration of a lens. Every measurement is one number, so the whole data path fits on a microcontroller. This makes the device a good case study for an embedded-systems course, because it combines real-time motor control, analog signal acquisition, multitasking, wireless communication and geometric data processing in one product. The project started with the TF-Luna LiDAR, and the experience that it cannot measure at 30–90 mm led to the infrared sensor used in the final design.

1.2. Objectives

The general objective is to design and build a working embedded 3D scanner around the ESP32-S3 and the Sharp GP2Y0A51SK0F, and to evaluate its accuracy and reliability. The specific objectives are:

- scan objects up to about 120 mm in diameter and 150 mm in height with 100 points per revolution (3.6°) and a 1 mm layer step;
- start automatically at power-on without any user input on the device, and finish by returning the Z axis to its start position;
- acquire the analog sensor signal reliably on GPIO18 even though ADC2 is shared with the Wi-Fi radio;
- transfer every finished layer to a PC over Wi-Fi with an integrity check, and keep scan data only in RAM (no flash writes);
- reconstruct a closed, printable STL model on the PC, with a dimensional error of ±1–2 mm on a 60 mm reference cube;
- verify the firmware and PC software systematically, with at least five test runs per program.

1.3. Subject and scope

The research object is a turntable-plus-Z-axis scanner with a single-point infrared triangulation sensor. The sensor face is 90 mm from the rotation axis, so the object surface must lie between 30 mm and 89 mm from the sensor (radius 1–60 mm). The reference test object is a 4 × 4 puzzle cube of 60 × 60 × 65 mm, wrapped in white A4 paper to give a matt, uniform surface. The scope is limited to:

- side surfaces that the horizontal beam can see; the top surface is closed by a flat cap and is not measured;
- matt, opaque, light-coloured surfaces; glossy, black or translucent materials are discussed but not fully compensated;
- a stationary machine on a stable desk, indoor lighting, and a PC within Wi-Fi range of the ESP32-S3;
- the hardware and software described in this report: `scan.cpp` (firmware), `pcrecieve.py` and `xyztostl.py` (PC).

1.4. Research methodology

The work combined four methods. A literature and datasheet review established the operating principle and limits of each component [3], [4], [5], [6].

Each hardware block was then tested on its own with a dedicated test program (A4988 and stepper test, Sharp sensor test, turntable-motion test). Because rare cases such as Wi-Fi loss or heavy ADC failures are hard to reproduce on the bench, the unmodified firmware was also compiled on a PC against stub Arduino, FreeRTOS and Wi-Fi libraries and driven by a simulated machine; this software-in-the-loop setup made such tests repeatable, for example with 36 % ADC failures. Finally, real objects were scanned and the results were compared with their measured dimensions.

1.5. Scientific and practical significance

Scientifically, the project quantifies how far a single low-cost infrared triangulation sensor can be pushed for 3D scanning. It shows that a lateral beam offset of only 14 mm turns a square section into a pinwheel shape, and it derives and validates a software correction for this offset. It also shows, by simulation, that the calibration table must be measured on the same surface and geometry as the scanned object, because the sensor response depends on incidence angle and surface material.

Practically, the scanner costs a fraction of a commercial device, can be built from 3D-printer parts, and produces STL files that open directly in slicers and CAD tools. The firmware structure (two pinned tasks sharing a RAM buffer, a framed TCP protocol, and robust ADC sampling next to Wi-Fi) is reusable in other ESP32-S3 data-acquisition projects.

CHAPTER 2: THEORETICAL BACKGROUND

2.1. 3D scanning technologies and point clouds

A 3D scanner samples the surface of an object as a set of points (x, y, z), called a point cloud. A mesh-building step then connects the points into triangles. Scanning methods are usually divided into contact and non-contact methods; non-contact methods are divided again into active methods, which project their own light, and passive methods, which use ambient light [3], [7]. Figure 2.1 shows this classification and where the present project fits.

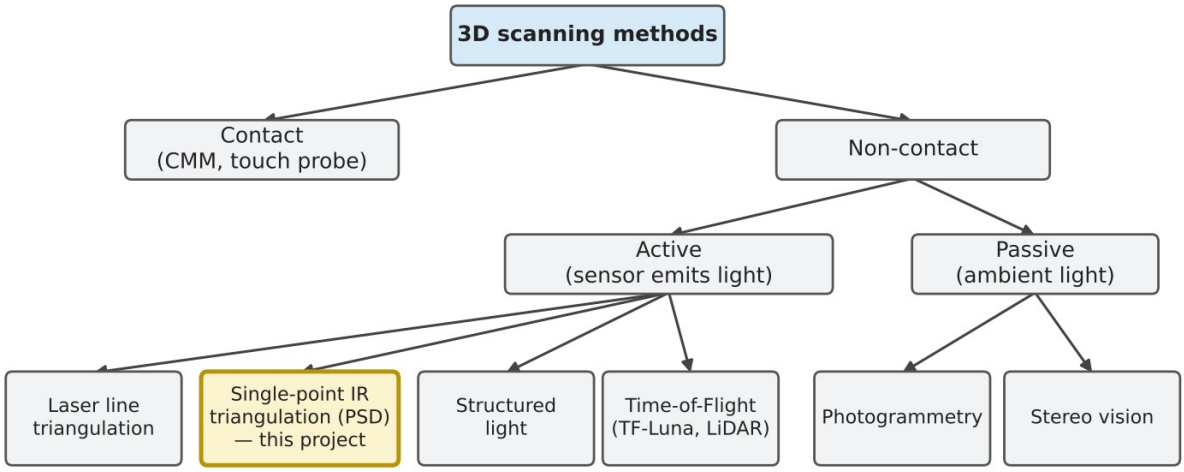


Figure 2.1. *Classification of 3D scanning methods; the highlighted block is the method used in this project*

Contact methods (coordinate measuring machines, touch probes) are very accurate but slow, and they can deform soft objects. **Triangulation** measures the angle at which a projected spot or line is seen by a detector placed at a known baseline from the emitter; it is accurate at short range and is used by laser-line scanners and by the Sharp sensor [3]. **Structured light** projects patterns onto the whole object and captures them with a camera. **Time of flight** measures the travel time or phase of reflected light; it works well at long range, but its absolute error is roughly constant (several centimetres for low-cost modules), so it is poorly suited to small objects [3], [8]. **Photogrammetry and stereo vision** reconstruct shape from several camera

images and need textured surfaces and heavy computation. Table 2.1 summarises these trade-offs.

Table 2.1. *Qualitative comparison of 3D scanning methods (compiled from [3], [7], [8])*

Method	Typical range	Accuracy at 5–10 cm	Cost	Main limitation
Contact (CMM)	workspace	µm	very high	slow, touches the object
Laser-line triangulation	0.1–1 m	0.1–0.5 mm	low–medium	needs camera and laser calibration
Single-point IR triangulation	2–15 cm (Sharp)	≈0.2–1 mm	very low	one point at a time; surface-dependent
Structured light	0.2–2 m	0.05–0.5 mm	medium–high	ambient light, shiny surfaces
Time of flight (TF-Luna)	0.2–8 m	±6 cm	low	blind zone below 0.2 m
Photogrammetry	any	depends on images	low	needs texture and heavy processing

2.2. Cylindrical-coordinate scanning principle

2.2.1. Geometric model

In a turntable scanner the sensor stays fixed in the horizontal plane while the object rotates. Figure 2.2 shows the geometry of one layer. The sensor face is at distance D from the rotation axis. At point index p the table has turned $\theta = p \times 3.6°$, the sensor measures distance d to the surface, and the radius of the surface point is taken as $r = D - d$. After a full revolution the Z axis lifts the sensor by Δz and the next layer starts.

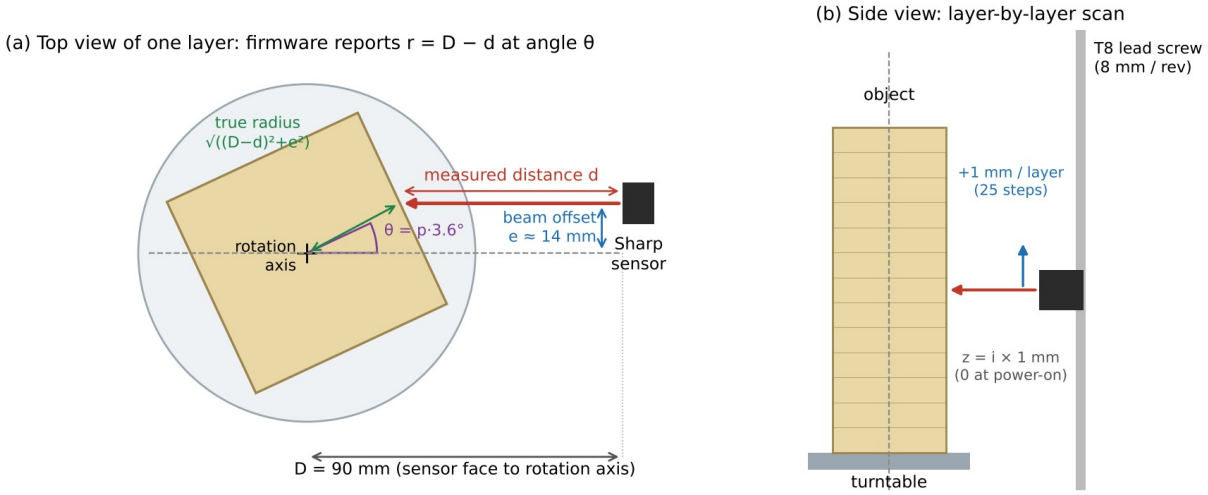


Figure 2.2. *Geometric model of the scanner: (a) top view of one layer with the beam offset e, (b) side view of the layer-by-layer scan*

2.2.2. Conversion to Cartesian coordinates

For layer i and point p, the firmware computes

$$r = D - d, \quad \theta = p \cdot 360° / N, \quad z = i \cdot \Delta z \tag{2.1}$$

$$x = r \cdot \cos\theta, \quad y = r \cdot \sin\theta \tag{2.2}$$

with N = 100 points per revolution and Δz = 1 mm. Equations (2.1) and (2.2) assume that the beam passes through the rotation axis. In the real machine the beam is displaced sideways by e ≈ 14.1 mm (Section 3.5). The true radius of the hit point is then $\sqrt{((D - d)^2 + e^2)}$ and its true angle is shifted by atan(e / (D − d)). Ignoring e makes flat faces appear rotated and bent, which produces the pinwheel shape analysed in Chapter 4; the PC software corrects it (Section 3.8).

2.2.3. Resolution and scanning volume

With 1.8° full steps and two steps per point, the angular resolution is 3.6°. The arc length between neighbouring points is r × 3.6° × π / 180, i.e. 1.9 mm at r = 30 mm (the middle of a cube face) and 2.6 mm at r = 42 mm (a cube corner). The vertical resolution equals the layer step, 1 mm (25 full steps of a T8 lead screw). The usable radius is 1–60 mm, limited by the first calibration point at 30 mm from the sensor, and the Z travel limit is 150 mm, giving at most 151 layers and 15 100 points per scan.

2.3. The Sharp GP2Y0A51SK0F infrared distance sensor

2.3.1. Triangulation measuring principle

The GP2Y0A51SK0F combines an infrared LED, a position-sensitive detector (PSD) and a signal-processing circuit in one package and outputs an analog voltage [4]. Figure 2.3 illustrates how it works. The LED, behind its own lens, sends a narrow beam forward. The object scatters part of the light back, and the receiver lens, mounted at a fixed baseline B from the emitter, focuses it onto the PSD. Because the reflected ray reaches the receiver at an angle that depends on distance, the light spot lands at a different position x on the PSD for every distance. From similar triangles, x = f · B / d, where f is the focal length of the receiver lens [9], [10]. A nearer object therefore gives a larger x and a higher output voltage.

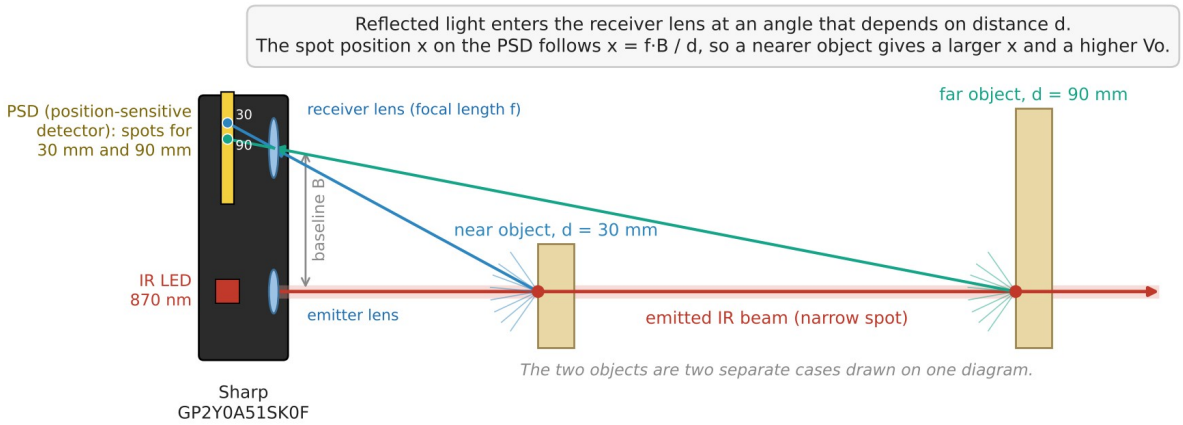


Figure 2.3. *Measuring principle of the Sharp GP2Y0A51SK0F: the reflected spot falls on a different position of the PSD for a near and a far object*

The PSD outputs two photocurrents whose ratio depends only on the spot position, not on its brightness [10]. This is why a triangulation sensor is, to first order, insensitive to the colour of the object: a darker object returns less light, but the spot stays in the same place. Since x is proportional to 1/d, the output voltage follows approximately

$$V_o \approx k / (d + b) + c \tag{2.3}$$

where k, b and c depend on the individual sensor and on the surface. Pololu describes the output as roughly linear in 1/distance [11]. Equation (2.3) explains the shape of

the measured curve in Figure 2.4: the voltage changes quickly at short distance and slowly at long distance.

2.3.2. Specifications and output characteristic

Table 2.2 lists the datasheet values that matter for this design. The sensor updates its output only once per measuring cycle of 16.5 ± 3.7 ms, so reading the ADC faster than this repeats the same value; this fact sets the sampling strategy of Section 2.4. Pololu recommends a capacitor of 10 µF or more across the supply close to the sensor, because the LED is pulsed and draws current in bursts [11].

Table 2.2. *Main specifications of the Sharp GP2Y0A51SK0F [4], [11]*

Parameter	Value
Measuring range	2–15 cm
Output type	analog voltage, rises as distance falls
Output at 15 cm	0.4 V typical (0.25–0.55 V)
Output change from 15 cm to 2 cm	1.65 V typical (1.35–1.95 V)
Supply voltage	4.5–5.5 V
Average supply current	12 mA typical, 22 mA maximum
Measuring cycle	16.5 ± 3.7 ms
LED wavelength	870 ± 70 nm
Package (datasheet outline)	27.0 × 10.8 × 12.0 mm

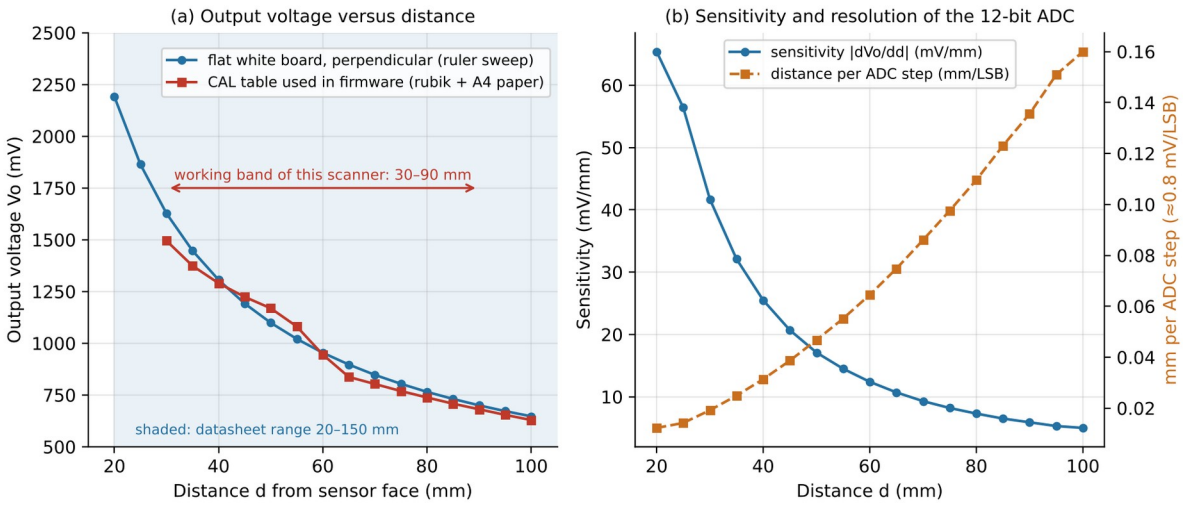


Figure 2.4. *(a) Measured output voltage of the sensor used in this project, on a flat white board and on the paper-wrapped cube; (b) sensitivity and the distance represented by one ADC step*

Figure 2.4(a) shows two curves measured with the scanner's own sensor. The blue curve was taken with a flat white board held perpendicular to the beam at 5 mm intervals on a ruler. The red curve is the calibration table used in the firmware; it was obtained on the paper-wrapped cube itself and is discussed in Section 4.3. Figure 2.4(b) shows that the sensitivity falls from about 40 mV/mm at 30 mm to about 6 mV/mm at 90 mm. With an ADC step of about 0.8 mV, one step corresponds to 0.02 mm at 30 mm but 0.14 mm at 90 mm, so noise in millivolts becomes much larger noise in millimetres as the object moves away from the sensor.

2.3.3. How the beam interacts with different surfaces

The insensitivity to colour holds only for matt (diffuse) surfaces seen at normal incidence. Sharp's application note lists mirrors, glass, glossy objects that are tilted, sharp boundaries between materials, object motion and strong ambient light as error sources, and recommends that material or colour boundaries run parallel to, and the direction of motion run perpendicular to, the line joining emitter and receiver [9]. Studies of IR and laser triangulation sensors confirm that response depends on reflectance and on incidence angle [12], [13]. Figure 2.5 shows the six situations met in this project.

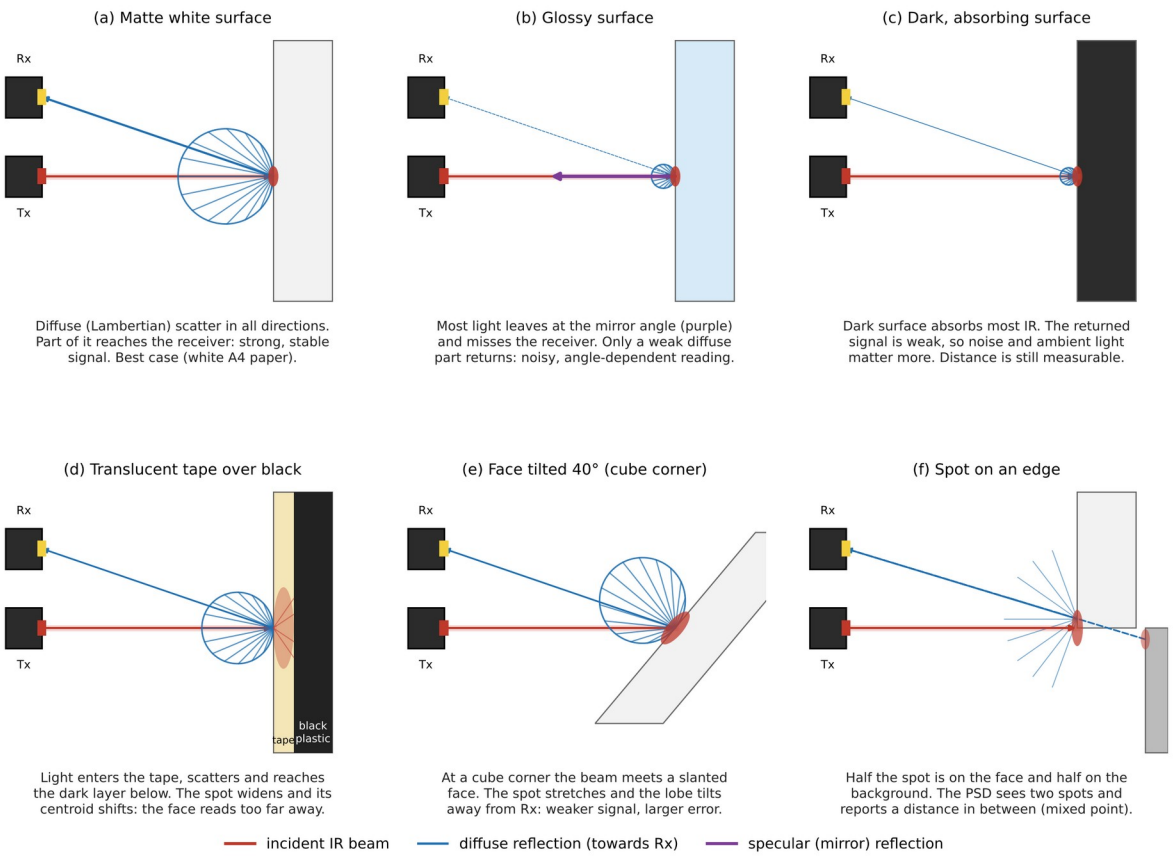


Figure 2.5. *How the infrared beam is reflected by different surfaces and what the receiver sees*

- **Matt white surface (a).** Light is scattered in all directions with a cosine-shaped (Lambertian) lobe. A large part reaches the receiver and the spot is small and well defined. This is why the test cube was wrapped in white A4 paper.
- **Glossy surface (b).** Most light leaves at the mirror angle. At normal incidence it returns towards the emitter, not the receiver, so only a weak diffuse part is detected; when the surface is tilted the strong reflection can even hit the receiver from the wrong direction.
- **Dark surface (c).** Most light is absorbed. The spot position is still correct, but the weaker signal is noisier.
- **Translucent tape over a dark body (d).** Light enters the tape, spreads sideways and is partly absorbed by the dark layer below. The spot becomes wide and its centre moves, so the surface reads too far away. This explains why the first scans with masking tape had faces pulled in by 11–15 mm.

– **Tilted face (e).** Near a cube corner the beam meets the face at a large angle. The spot is stretched along the face and the reflected lobe turns away from the receiver, so the signal is weaker and the error larger. The PC software therefore removes points whose incidence angle exceeds 60°.
– **Spot on an edge (f).** When the spot falls half on the object and half on the background, the PSD sees two spots and reports a distance in between. Such mixed points appear as isolated outliers and are removed by the firmware's neighbour filter.

2.3.4. Why the TF-Luna was replaced

The TF-Luna is a single-point time-of-flight LiDAR with a range of 0.2–8 m on a 90 % reflective target, a blind zone below 0.2 m, an accuracy of ±6 cm from 0.2 to 3 m, and a resolution of 1 cm [8]. In this scanner the surface is only 30–90 mm from the sensor face, completely inside the blind zone, and the ±6 cm accuracy band is as wide as the whole working range. A larger sensor distance would make the machine large and would still give centimetre-level errors. The Sharp sensor is specified exactly for 2–15 cm, costs less, and needs only one analog pin. Table 2.3 compares the two sensors.

Table 2.3. *Comparison between the TF-Luna and the Sharp GP2Y0A51SK0F for this scanner [8], [4]*

Criterion	TF-Luna (first prototype)	Sharp GP2Y0A51SK0F (final)
Principle	time of flight (modulated light)	triangulation on a PSD
Range	0.2–8 m (blind zone < 0.2 m)	2–15 cm
Accuracy / resolution at short range	±6 cm, 1 cm	≈0.2–1 mm after averaging (Section 4.2)
Interface	UART or I²C, 2 pins	analog voltage, 1 ADC pin
Update rate	1–250 Hz	≈60 Hz (16.5 ms cycle)
Needs calibration	no (outputs millimetres)	yes (voltage → distance table)
Surface sensitivity	low	medium to high (gloss, translucency, angle)
Usable at 30–90 mm	no	yes

2.4. ESP32-S3, FreeRTOS and analog acquisition

2.4.1. The ESP32-S3 microcontroller

The ESP32-S3 is a system-on-chip with two Xtensa LX7 cores running at up to 240 MHz, 512 KB of SRAM, 2.4 GHz Wi-Fi (802.11 b/g/n) and Bluetooth LE, and two 12-bit SAR ADCs with up to 20 channels [5]. The project uses the ESP32-S3-DevKitC-1 board, programmed with the Arduino core in PlatformIO. Table 2.4 lists the features that the design relies on.

Table 2.4. *ESP32-S3 features used in this project [5], [14], [15]*

Feature	Value	Use in the scanner
CPU	2 × Xtensa LX7, up to 240 MHz	core 1: motion and sensing; core 0: Wi-Fi
SRAM	512 KB	scan buffer g_r (≈30 KB) kept in RAM
Wi-Fi	802.11 b/g/n, 2.4 GHz	SoftAP 'ScannerESP32', TCP server on port 5000
ADC	2 × 12-bit SAR, 20 channels	GPIO18 = ADC2_CH7 reads the Sharp output
Attenuation 11 dB	≈150–3100 mV input range	covers the 0.6–2.2 V sensor output
GPIO	45 programmable pins	6 outputs for two A4988 drivers

2.4.2. FreeRTOS tasks on two cores

ESP-IDF, on which the Arduino core is built, uses an SMP version of FreeRTOS (based on FreeRTOS 10.5.1). A task can be pinned to a core with `xTaskCreatePinnedToCore`; core 0 is used by the Wi-Fi and protocol stack, and core 1 is normally free for application code [15]. Pinning the motion task to core 1 keeps step timing and ADC sampling away from Wi-Fi interrupts, while the communication task on core 0 sits next to the network stack. Tasks exchange data

through memory; in this firmware the shared counters are C++ `std::atomic` variables, so neither task needs a lock.

2.4.3. ADC2 and Wi-Fi coexistence

On the ESP32-S3, ADC2 is also used by the Wi-Fi driver. ESP-IDF arbitrates between the two, and a one-shot read may fail or time out, in which case the result is invalid [16]. The printed circuit board of this project routes the sensor to GPIO18, which is ADC2_CH7, so the firmware must live with this restriction. In this project, a failed conversion read through the Arduino function `analogReadMilliVolts()` [14] was observed as a value near 0 mV. Because the Sharp output never falls below about 0.4 V, any reading under 100 mV can be safely recognised as a failed conversion and discarded.

2.4.4. From voltage to distance

Figure 2.6 shows the complete measurement chain. The sensor output enters GPIO18 with 11 dB attenuation, giving a 0–3.1 V range at 12-bit resolution (about 0.8 mV per step). `analogReadMilliVolts()` applies the chip's factory (eFuse) calibration and returns millivolts. For every point the firmware takes 24 valid samples 1 ms apart, discards failed conversions, sorts the samples, rejects the point if the spread is too large, and averages the middle half (a trimmed mean). The result Vo is converted to distance by linear interpolation in the calibration table, and the radius is r = D − d.

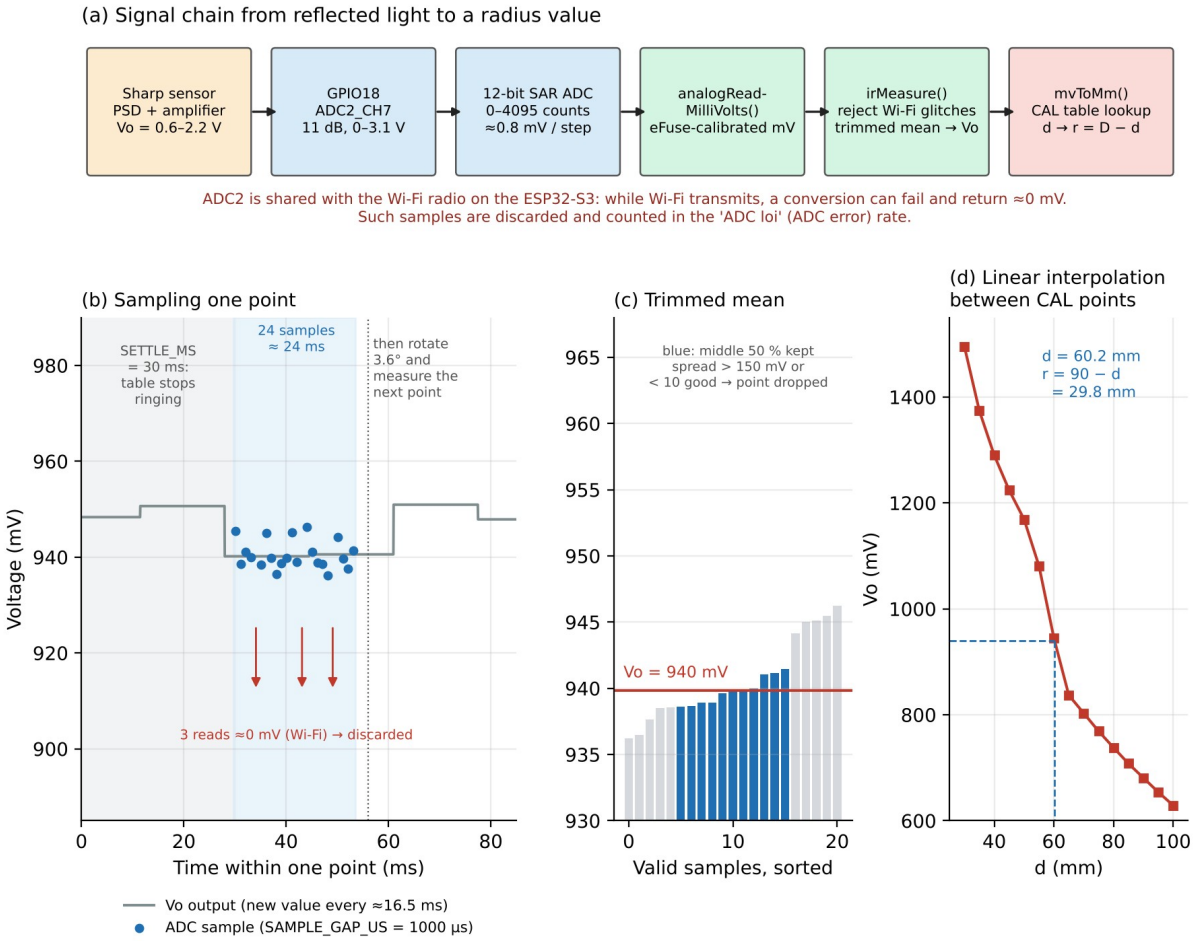


Figure 2.6. *Measuring distance through the ADC: (a) signal chain, (b) timing of one point, (c) trimmed mean, (d) interpolation in the calibration table*

Two remarks follow from Figure 2.6(b). First, the 30 ms wait before sampling lets the turntable stop ringing after it moves. Second, the 24 ms read window covers only one or two sensor updates (16.5 ms each), so averaging reduces ADC noise but not much of the sensor's own cycle-to-cycle noise. Spreading the samples over a longer window would average more independent sensor readings; Section 4.4 quantifies this trade-off between accuracy and scan time.

2.5. Stepper motors, the A4988 driver and the lead screw

A NEMA17 hybrid stepper with 1.8° steps makes 200 full steps per revolution. The A4988 driver accepts STEP, DIR and EN logic signals, regulates the coil current with a chopper, and supports full, half, 1/4, 1/8 and 1/16 microstepping selected by MS1–MS3 [6]. Each rising edge on STEP advances the motor one step; the minimum high and low pulse widths are 1 µs, and the current limit is set by VREF according to I

= VREF / (8 · R_S) [6]. With MS1–MS3 left unconnected, as in this design, the driver runs in full-step mode, which gives the largest torque per step and the simplest angle bookkeeping (two pulses = 3.6°).

Full stepping has a drawback: at every step the rotor overshoots and rings before settling, and at some speeds resonance causes vibration and noise. A loaded turntable behaves like a lightly damped mass–spring system with a natural period of several tens of milliseconds. The firmware therefore sends the two pulses of each 3.6° move about half a ringing period apart (`TABLE_STEP_GAP_US` = 22 000 µs). The second step then starts while the first oscillation is swinging back, and the two oscillations largely cancel, as illustrated in Figure 2.7. Larger moves, such as lifting the Z axis by 25 steps or returning it home, use a linear speed ramp from a 2.5 ms half-period down to the cruise speed so that the motor does not lose steps.

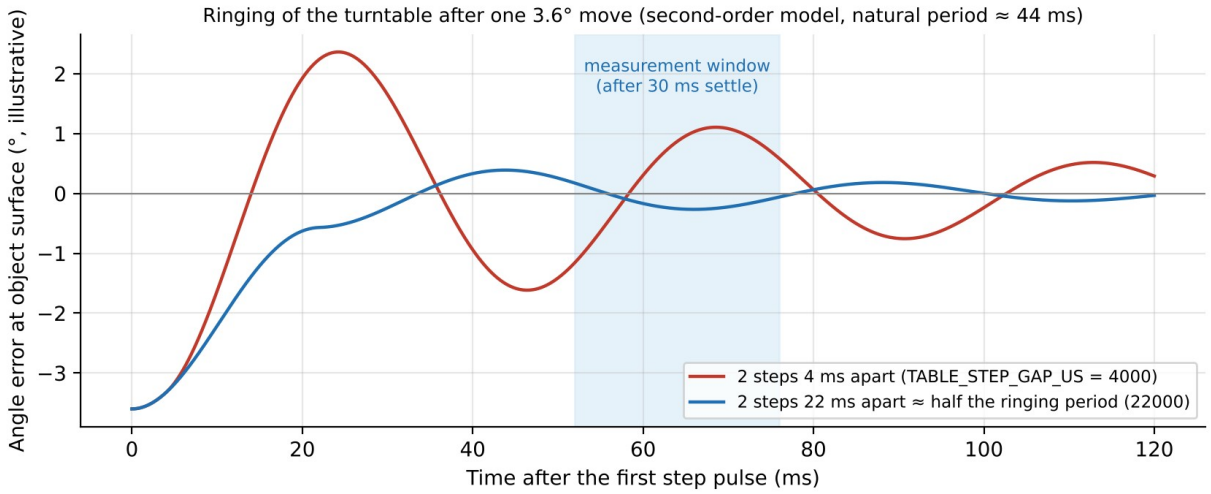


Figure 2.7. *Illustrative turntable ringing after one 3.6° move: two steps half a period apart cancel most of the oscillation before the measurement window*

The Z axis uses a T8 lead screw with four starts, a 2 mm pitch and an 8 mm lead, so one motor revolution lifts the carriage 8 mm and one full step lifts it 0.04 mm. A 1 mm layer is exactly 25 steps. Layer steps that are not multiples of 0.04 mm would accumulate rounding errors, which is why the firmware computes all Z moves in whole steps.

2.6. Wi-Fi SoftAP and TCP communication

In SoftAP mode the ESP32-S3 creates its own Wi-Fi network, gives itself the address 192.168.4.1 and assigns addresses to joining stations [17]. No router is needed, which keeps the setup portable and avoids depending on a campus network. On top of Wi-Fi the scanner uses TCP, which delivers a reliable, ordered byte stream with retransmission and checksums. TCP does not know where a scan begins or ends, so the application adds its own framing: a START line with the scan identifier, one text line per point, and an END line with the point count, which the PC answers with ACK or NACK. A blank line sent every 2 s acts as a heartbeat so that the PC can detect a silent link.

Keeping the data in RAM rather than flash was a deliberate requirement. Flash memory has limited write endurance and writing it while Wi-Fi is active adds latency; a full scan needs only about 30 KB of RAM, well within the 512 KB available, and the TCP framing lets the ESP32-S3 resend the whole scan from RAM if the PC connects late or the link drops.

CHAPTER 3: SYSTEM DESIGN

3.1. System requirements

Table 3.1. *Functional and non-functional requirements*

ID	Requirement	How it is met
F1	Rotate the object in 3.6° steps and lift the sensor 1 mm per layer	two A4988 + NEMA17, full step, T8 lead screw
F2	Measure distance 30–90 mm with sub-millimetre repeatability	Sharp GP2Y0A51SK0F, 24-sample trimmed mean
F3	Start automatically and stop at the top of the object	scan starts at power-on; empty-layer detection
F4	Send data wirelessly while scanning	SoftAP + TCP, one layer at a time
F5	Detect lost or incomplete transfers	START/END framing, point count, ACK/NACK, heartbeat
F6	Produce a closed STL model	xyztostl.py grid mesh with top and bottom caps
N1	No user interface on the device	push button removed; RST key restarts a scan
N2	Do not write scan data to flash	RAM buffer only; WiFi.persistent(false)
N3	Robust ADC reading next to Wi-Fi	failed reads discarded, failure rate reported
N4	Low cost, common parts	3D-printer parts, < 1 kg, single 12 V supply

3.2. Overall block diagram

Figure 3.1 shows the complete system. The scanner unit contains the power supply, the ESP32-S3, two A4988 drivers with their motors, and the Sharp sensor. The PC unit contains the receiver program, the point file and the mesh builder. The only link between the two units is Wi-Fi; the USB serial port is used for programming and for a human-readable log.

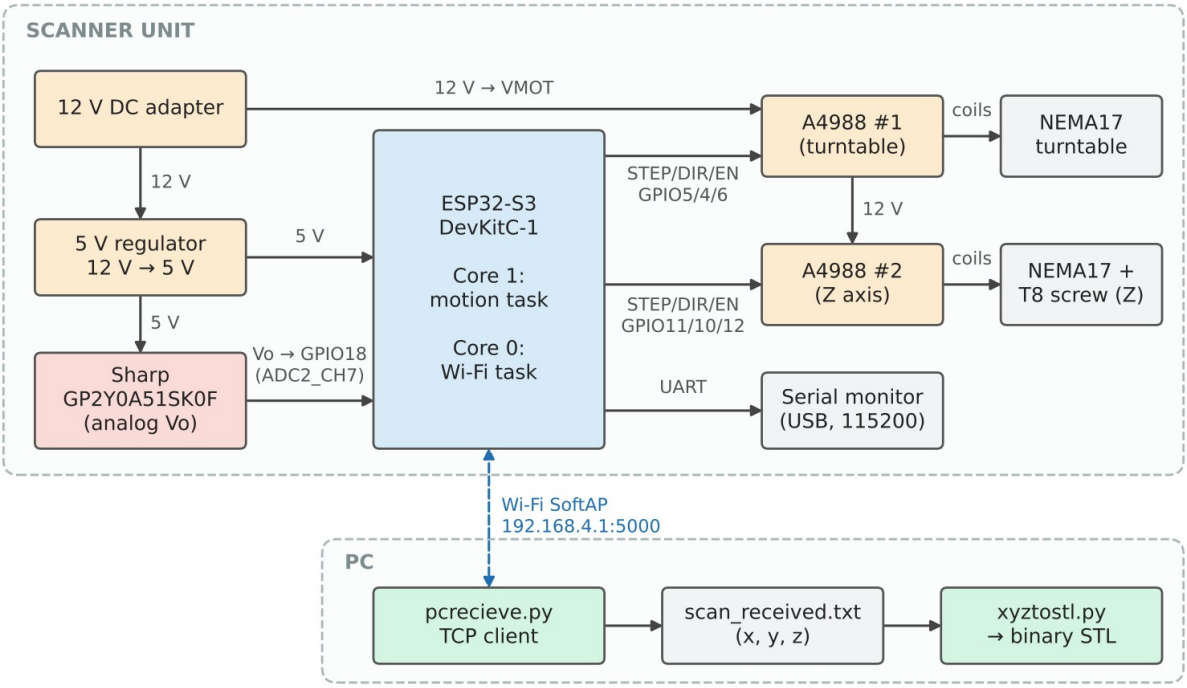


Figure 3.1. *Overall block diagram of the 3D scanning system*

3.3. Detailed design of each block

3.3.1. Power block

A 12 V DC adapter, connected to the 12V_SOURCE terminal, supplies the motor inputs (VMOT) of both A4988 drivers; a 100 µF electrolytic capacitor (C1) on the 12 V rail absorbs the voltage spikes caused by current chopping. A separate 5 V module feeds the 5V_SOURCE terminal, with a 47 µF capacitor (C2), and powers the ESP32-S3 board (5V0 pin) and the Sharp sensor. The board's own regulator provides 3.3 V, decoupled by a 100 nF capacitor (C4), which also supplies the logic side (VDD) of the two drivers. All grounds are joined. At boot the firmware prints the reset reason and warns if it was a brown-out, which helps diagnose an undersized supply.

3.3.2. Central processing block

The ESP32-S3-DevKitC-1 runs the firmware `scan.cpp`. It generates six digital outputs for the drivers, reads one analog input, hosts the Wi-Fi access point and prints a one-line summary for every layer on the serial port at 115 200 baud.

3.3.3. Sensor block

The Sharp sensor is powered from 5 V and its output Vo is connected directly to GPIO18. The maximum output, about 2.2 V at 20 mm, lies inside the 11 dB ADC range, so no voltage divider is needed. A capacitor of at least 10 µF is placed across the sensor supply. GPIO17, which carried the TF-Luna's second UART line on the existing PCB, is configured as a high-impedance input so that the old trace cannot disturb anything.

3.3.4. Motor block

Each A4988 drives one NEMA17. MS1–MS3 are left open (full step), RESET is tied to SLEEP so the driver is always awake, and the current limit is set with the on-board potentiometer. The turntable driver's EN is released at the end of a scan so the table can be turned by hand; the Z driver stays enabled so that the sensor carriage holds its position.

3.3.5. Communication block

The communication block is implemented in software on core 0: a SoftAP named ScannerESP32 (password scanner123) and a TCP server on port 5000. The PC joins the network and runs `pcrecieve.py`, which connects as a TCP client.

3.4. Wiring diagram and pin allocation

Figure 3.2 and Table 3.2 give the final wiring. Compared with the first prototype, the TF-Luna and its two UART wires were replaced by the three-wire Sharp sensor, and the push button on GPIO7 with its pull-down resistor was removed from the official design. The scan now starts automatically when the board is powered; pressing the RST key on the board starts a new scan.

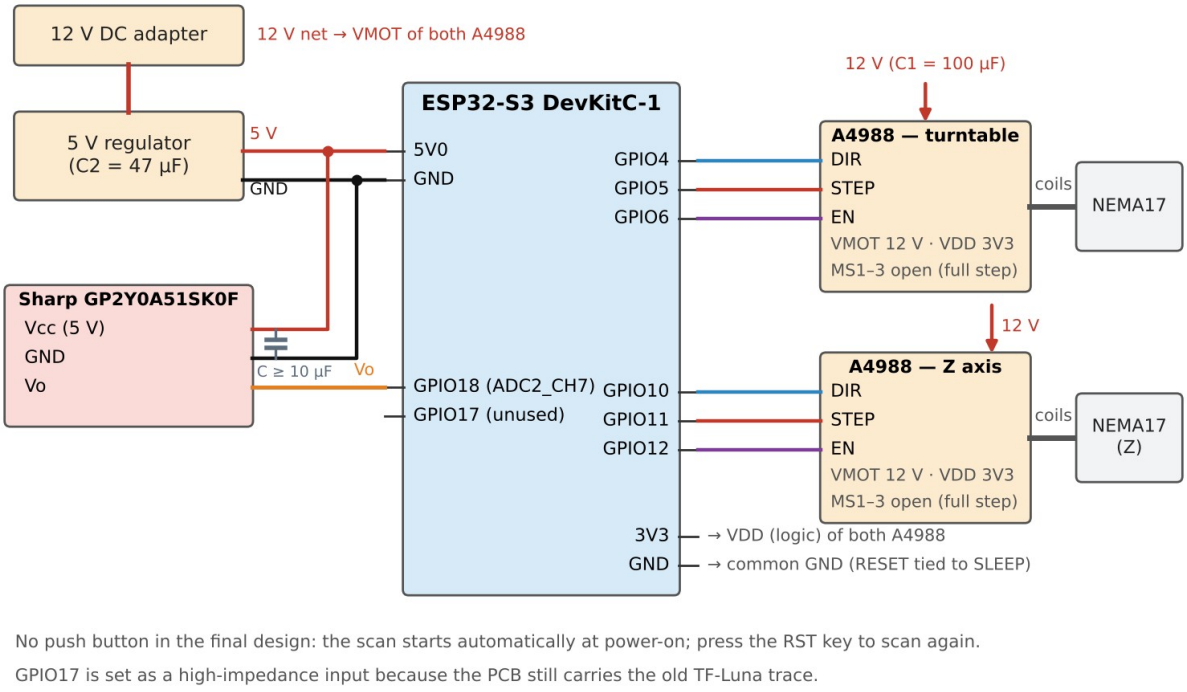


Figure 3.2. *Wiring between the ESP32-S3, the two A4988 drivers, the Sharp sensor and the power supply*

Table 3.2. *ESP32-S3 pin allocation (as defined in scan.cpp)*

ESP32-S3 pin	Connected to	Constant in code	Function
GPIO4	A4988 #1 DIR	PIN_TABLE_DIR = 4	turntable direction (TABLE_DIR_LEVEL = HIGH)
GPIO5	A4988 #1 STEP	PIN_TABLE_STEP = 5	one rising edge = one 1.8° step
GPIO6	A4988 #1 EN	PIN_TABLE_EN = 6	LOW = hold, HIGH = released after the scan
GPIO10	A4988 #2 DIR	PIN_Z_DIR = 10	HIGH = up (Z_UP_LEVEL)
GPIO11	A4988 #2 STEP	PIN_Z_STEP = 11	25 steps = 1 mm
GPIO12	A4988 #2 EN	PIN_Z_EN = 12	always LOW (Z holds position)
GPIO18 (ADC2_CH7)	Sharp Vo	PIN_IR = 18	analog input, 12-bit, 11 dB attenuation
GPIO17	not used	PIN_IR_UNUSED = 17	input, high impedance (old TF-Luna trace)

ESP32-S3 pin	Connected to	Constant in code	Function
5V0, 3V3, GND	5 V module, A4988 VDD, common ground	—	power

The control board was drawn as a schematic (Figure 3.3) and laid out as a PCB (Figure 3.4). U1 is the ESP32-S3-DevKitC-1, U2 and U3 are the A4988 carriers, and J1 and J2 are the four-pin motor terminals. The two headers kept from the first prototype are reused for the new sensor: the header labelled TF_LUNA_SOURCE provides 5 V and GND to the Sharp sensor, and the header labelled TF_LUNA_UART, wired to GPIO17 and GPIO18, now carries the sensor output Vo on its GPIO18 pin. GPIO17 is left unconnected and configured as an input in the firmware.

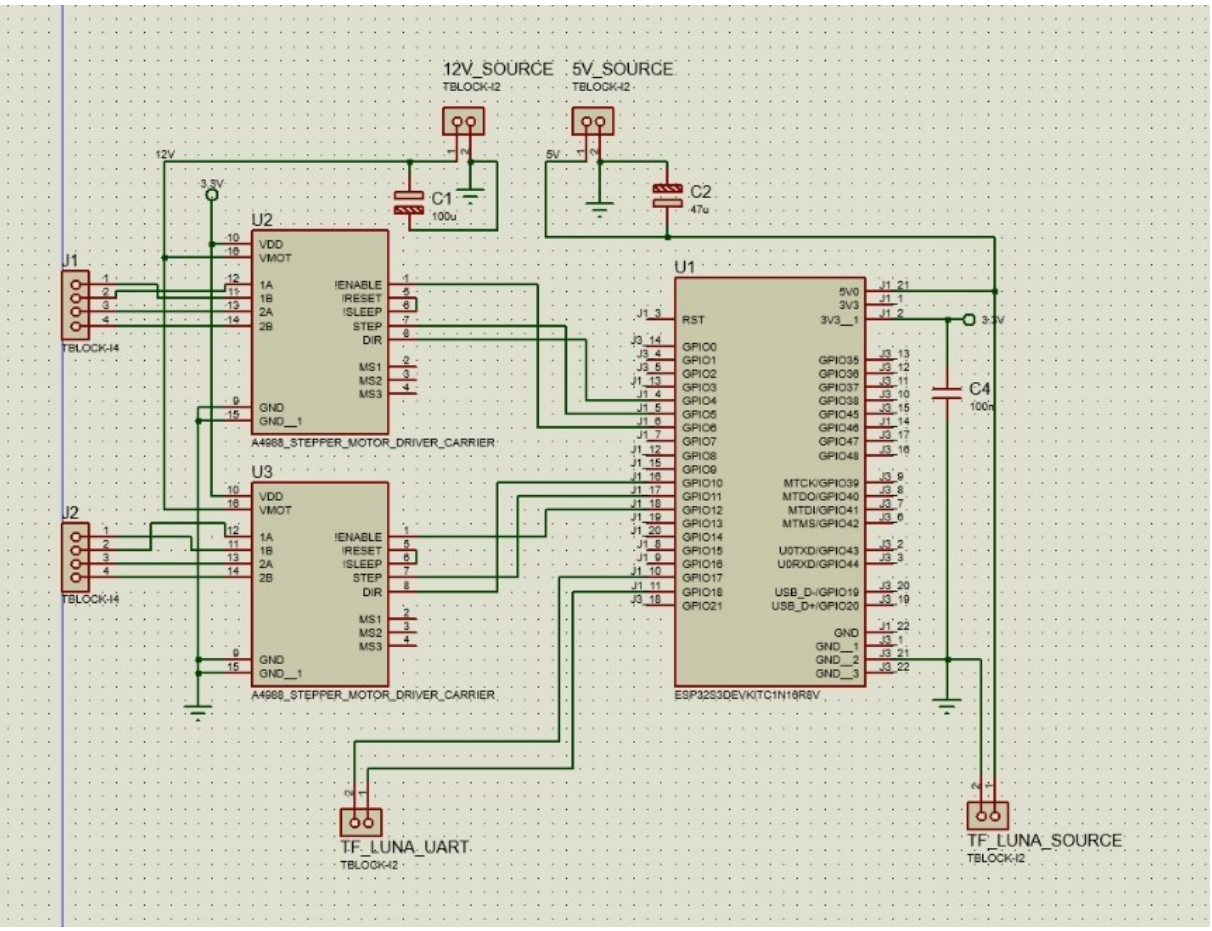


Figure 3.3. *Circuit schematic of the control board (the TF-Luna headers are reused for the Sharp sensor)*

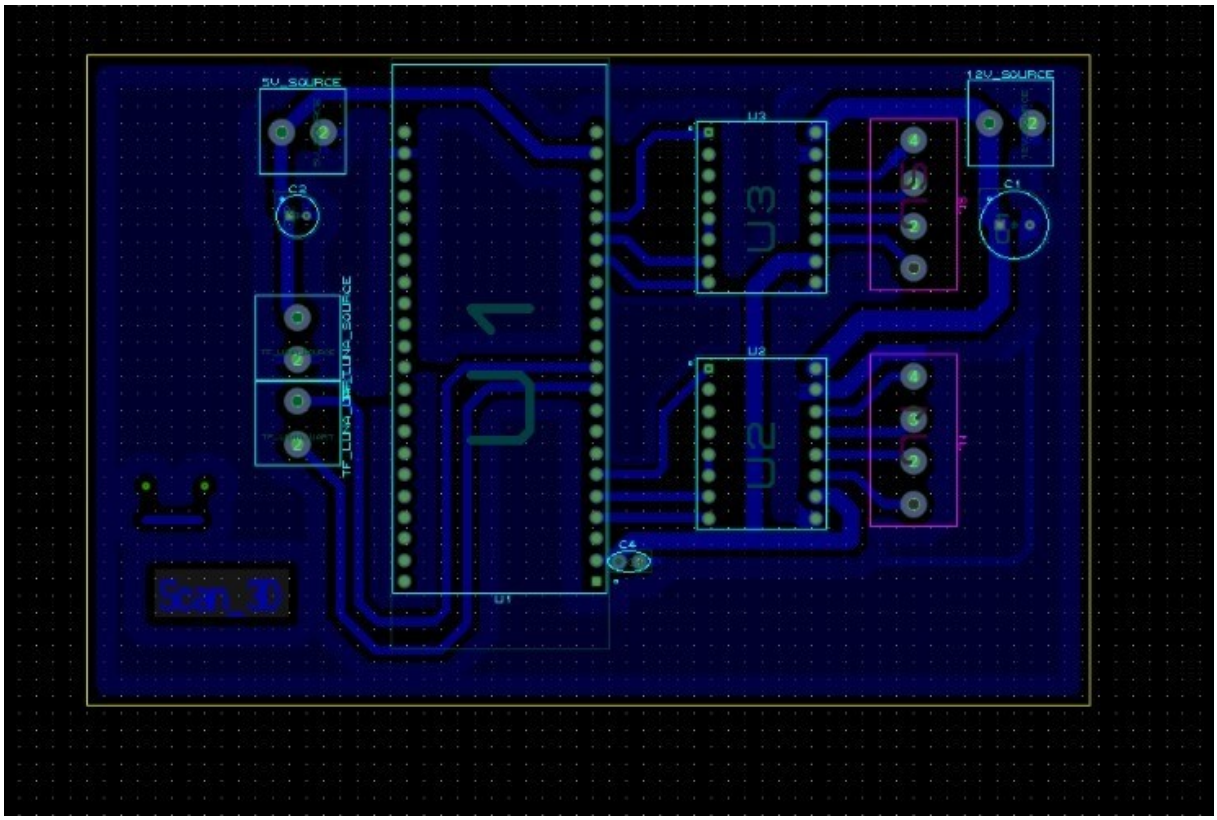


Figure 3.4. *PCB layout of the control board*

3.5. Mechanical design and geometric parameters

The mechanism reuses 3D-printer parts: a printed turntable mounted directly on the turntable motor shaft, two vertical guide rods, and a printed carriage that holds the sensor and rides on the T8 lead screw. The sensor's emitter–receiver line is vertical, so the direction of motion of the surface (horizontal) is perpendicular to it and the vertical edges of a box-shaped object are parallel to it, as recommended by Sharp [9]. Table 3.3 lists the geometric parameters. The beam offset e appears because the infrared emitter sits beside the centre line of the sensor carriage; it was estimated from photographs at about 13.5 mm and determined from scan data at 14.1 mm (Section 4.3). Instead of rebuilding the carriage, the offset is corrected in software.

All structural parts were modelled in CAD and 3D-printed (Figure 3.5): the base that holds the turntable motor, the round turntable, the vertical frame with the two guide rods and the lead screw, and the sensor carriage with its nut holder.

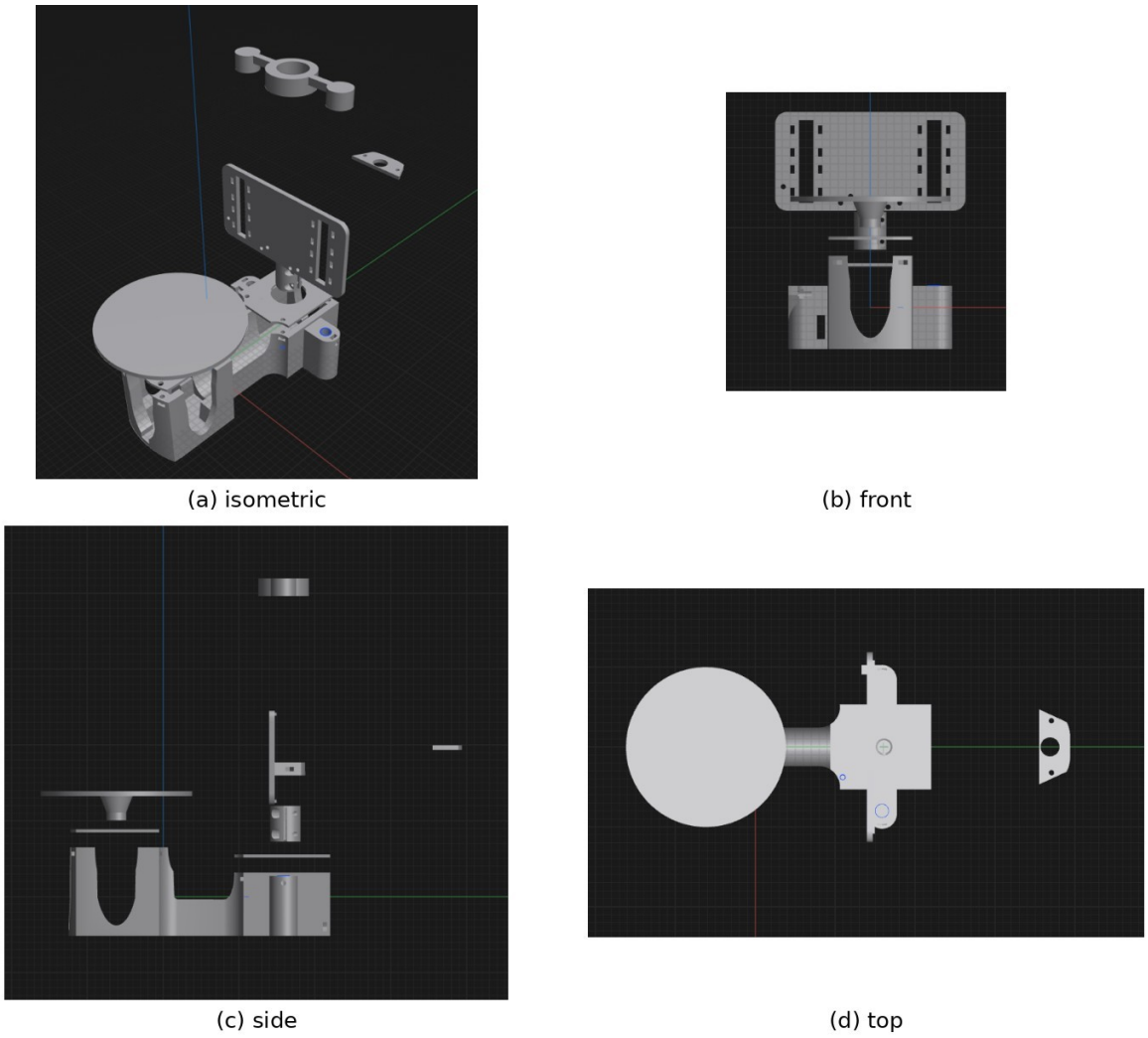


Figure 3.5. *CAD model of the 3D-printed frame: (a) isometric, (b) front, (c) side and (d) top views*

Table 3.3. *Geometric parameters of the scanning mechanism*

Parameter	Symbol / constant	Value
Sensor face to rotation axis	D, DIST_TO_CENTER_MM	90 mm (measured with a ruler)
Lateral beam offset	e, --beam-offset	14.1 mm
Usable radius	MIN_RADIUS_MM … MAX_RADIUS_MM	1–60 mm
Angle step	POINT_ANGLE_DEG	3.6° (2 full steps)
Layer step	LAYER_MM	1.0 mm (25 full steps)
Z travel limit	Z_TRAVEL_LIMIT_MM	150 mm (151 layers)

Parameter	Symbol / constant	Value
Beam height at z = 0	--z-day	≈10 mm above the table

3.6. Embedded software design

3.6.1. Configuration parameters

All tunable values are constants at the top of `scan.cpp`, so the behaviour of the machine can be changed without touching the logic. Table 3.4 explains each group. Compile-time checks (`static_assert`) stop the build if the angle step is not a multiple of 1.8°, or if the calibration table is not strictly ordered (mV decreasing, mm increasing).

Table 3.4. *Main configuration parameters of the firmware*

Group	Constant	Value	Meaning
Wi-Fi	AP_SSID / AP_PASS / PC_PORT	ScannerESP32 / scanner123 / 5000	network name, password and TCP port
Wi-Fi	HEARTBEAT_MS, ACK_WAIT_MS	2000 ms, 3000 ms	keep-alive period; wait for ACK after END
Mechanics	MOTOR_STEP_DEG, MICROSTEP, LEAD_SCREW_MM	1.8°, 1, 8 mm	200 steps/rev, full step, 25 steps/mm
Scan	POINT_ANGLE_DEG, LAYER_MM	3.6°, 1.0 mm	100 points per revolution, layer step
Geometry	DIST_TO_CENTER_MM, MIN_RADIUS_MM	90 mm, 1 mm	D and smallest accepted radius
Calibration	CAL_MV[], CAL_MM[]	15 points, 1495→629 mV for 30→100 mm	voltage → distance table
Measurement	SETTLE_MS	30 ms	wait after each table move
Measurement	SAMPLES, SAMPLE_GAP_US	24, 1000 µs	samples per point and spacing
Measurement	BAD_READ_MV, MIN_GOOD_SAMPLES, SPREAD_MAX_MV	100 mV, 10, 150 mV	failed-read threshold and point validity

Group	Constant	Value	Meaning
Start-up	WARMUP_MS	1000 ms	drivers hold the motors before scanning
Stopping	EMPTY_RATIO, EMPTY_LAYERS_TO_STOP, NO_OBJECT_LAYERS	2 %, 1, 5	end of object / no object detection
Filtering	OUTLIER_DIST_MM, OUTLIER_NEIGHBOURS	5 mm, 2	neighbour median filter in each layer
Motors	TABLE_STEP_GAP_US	22 000 µs	gap between the 2 steps of a point
Motors	Z_UP_HALF_US, Z_RETURN_HALF_US, START_HALF_US	1000, 800, 2500 µs	step half-periods: lift, return, ramp start
Motors	Z_RAMP_STEPS, Z_SETTLE_MS	100 steps, 30 ms	ramp length and wait after a lift
Transfer	TX_WAIT_MS	2000 ms	wait for the PC to receive a layer

3.6.2. Task architecture and code block diagram

Figure 3.6 is the block diagram of the main scan code. `setup()` configures the pins and the ADC (12-bit, 11 dB), creates a scan identifier from the microsecond timer, waits 1 s while the drivers energise the motors, prints the reset reason and starts two tasks. The motion task (core 1, priority 2, 8 KB stack) performs the scan. The communication task (core 0, priority 1, 6 KB stack) calls `commStep()` every 10 ms. The tasks never call each other: the motion task writes each finished layer into the shared array `g_r` and increments `g_layersDone`; the communication task sends every layer it has not sent yet. Wi-Fi problems therefore never stop the scan, and a slow scan never blocks Wi-Fi.

Listing 3.1 shows how the two tasks are created. `setup()` pins the communication task to core 0 and the motion task to core 1, and `loop()` deletes itself

because all work is done inside the tasks. The communication task is a simple loop that calls `commStep()` and then sleeps for 10 ms, so it never blocks the core.

Listing 3.1. *Creating the two FreeRTOS tasks (scan.cpp, shortened)*

```
void setup() {
  ...
  analogReadResolution(12);
  analogSetPinAttenuation(PIN_IR, ADC_11db);
  ...
  xTaskCreatePinnedToCore(commTask,   "comm",   6144, nullptr, 1, nullptr, 0);
  xTaskCreatePinnedToCore(motionTask, "motion", 8192, nullptr, 2, nullptr, 1);
}

void loop() {
  vTaskDelete(nullptr);
}

void commTask(void*) {
  commSetup();
  for (;;) {
    commStep();
    vTaskDelay(pdMS_TO_TICKS(10));
  }
}
```

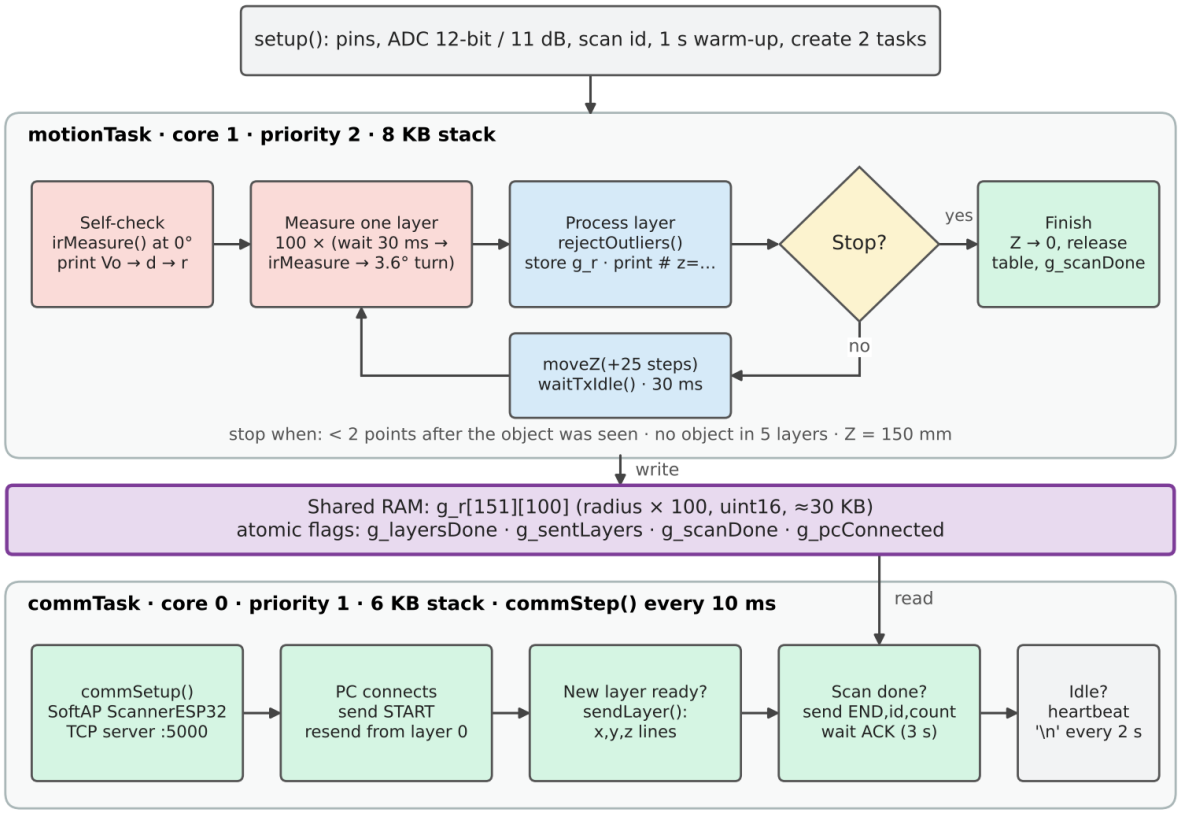


Figure 3.6. *Block diagram of the main scan firmware: two FreeRTOS tasks connected only through shared RAM*

Table 3.5 lists the function blocks and how they call each other.

Table 3.5. *Function blocks of scan.cpp*

Block	Function	What it does	Called by
Start-up	setup(), loop()	pins, ADC, warm-up, create tasks; loop() deletes itself	Arduino core
Measurement	irMeasure()	up to 72 reads for 24 good samples, spread check, trimmed mean	motionTask, runScan
Measurement	mvToMm(), radiusFrom()	CAL interpolation; r = D − d within 1–60 mm	irMeasure, runScan
Filtering	rejectOutliers(), medianSmall()	drop isolated points and points > 5 mm from the neighbour median	runScan
Motors	rotateTable()	two step pulses 22 ms apart	runScan
Motors	moveZ(), moveSteps(), rampHalfUs()	ramped Z moves, limited to 0–150 mm	runScan, motionTask
Control	runScan()	layer loop, layer summary, stop decision, Z lift	motionTask
Control	motionTask()	self-check, runScan, return home, release table, final message	setup
Control	waitTxIdle()	if a PC is connected, wait up to 2 s for the layer to be sent	runScan
Wi-Fi	commSetup(), commStep()	SoftAP, server, START, layers, END/ACK, heartbeat	commTask
Wi-Fi	sendLayer(), putLine(), flushTx()	format x,y,z lines, 1400-byte transmit buffer	commStep
Wi-Fi	dropPc(), resetConn()	close the client and restart the transfer from layer 0	commStep

3.6.3. Motion task and scan loop

The motion task first measures one point at 0° and prints Vo, d and r, which is a quick check that the sensor is connected and the object is inside the calibrated range. `runScan()` then repeats the following for every layer:

1. for each of the 100 points: wait 30 ms (`SETTLE_MS`), call `irMeasure()`, convert to a radius, and rotate the table by 3.6°;

2. run `rejectOutliers()` on the 100 radii; store them in `g_r` as unsigned 16-bit values in units of 0.01 mm (0 = no point);
3. increment `g_layersDone` and print a summary line: height, number of valid points, median Vo, median radius, ADC failure rate and layer time;
4. decide whether to stop (below); otherwise lift Z by 25 steps, wait until the PC has received the layer (at most 2 s, only if a PC is connected) and wait 30 ms for the carriage to settle.

The scan stops in three cases: a layer has fewer than 2 valid points after the object has been seen (the beam has passed the top), no object is found in the first five layers, or the next lift would exceed 150 mm. After the scan, Z returns to 0 with a faster ramp, the turntable driver is released and `g_scanDone` is set.

Listing 3.2 shows how one 3.6° move is made. In full-step mode the two STEP pulses are separated by `TABLE_STEP_GAP_US` (22 ms), about half a ringing period, so that the oscillation caused by the first step is cancelled by the second one (Section 2.5).

Listing 3.2. *Turntable move with timed steps, rotateTable()*

```
void rotateTable(long steps) {
  digitalWrite(PIN_TABLE_DIR, TABLE_DIR_LEVEL);
  delayMicroseconds(5);
  if (MICROSTEP > 1 || steps > SHORT_MOVE_STEPS) {
    moveSteps(PIN_TABLE_STEP, steps, TABLE_HALF_US, TABLE_RAMP_STEPS);
    return;
  }
  for (long i = 0; i < steps; i++) {
    if (i) delayMicroseconds(TABLE_STEP_GAP_US);
    digitalWrite(PIN_TABLE_STEP, HIGH);
    delayMicroseconds(10);
    digitalWrite(PIN_TABLE_STEP, LOW);
  }
}
```

3.6.4. Point measurement and filtering

Figure 3.7 is the flowchart of `irMeasure()`. Samples below 100 mV are failed ADC2 conversions and are only counted. Valid samples are inserted into a sorted array. If fewer than 10 valid samples are obtained in 72 reads, or if the spread between the 1/8 and 7/8 quantiles exceeds 150 mV (typically when the spot falls on an edge), the point is left empty. Otherwise Vo is the mean of the middle 50 % of the samples, d is interpolated from the calibration table, and r = D − d is kept if it lies in 1–60 mm.

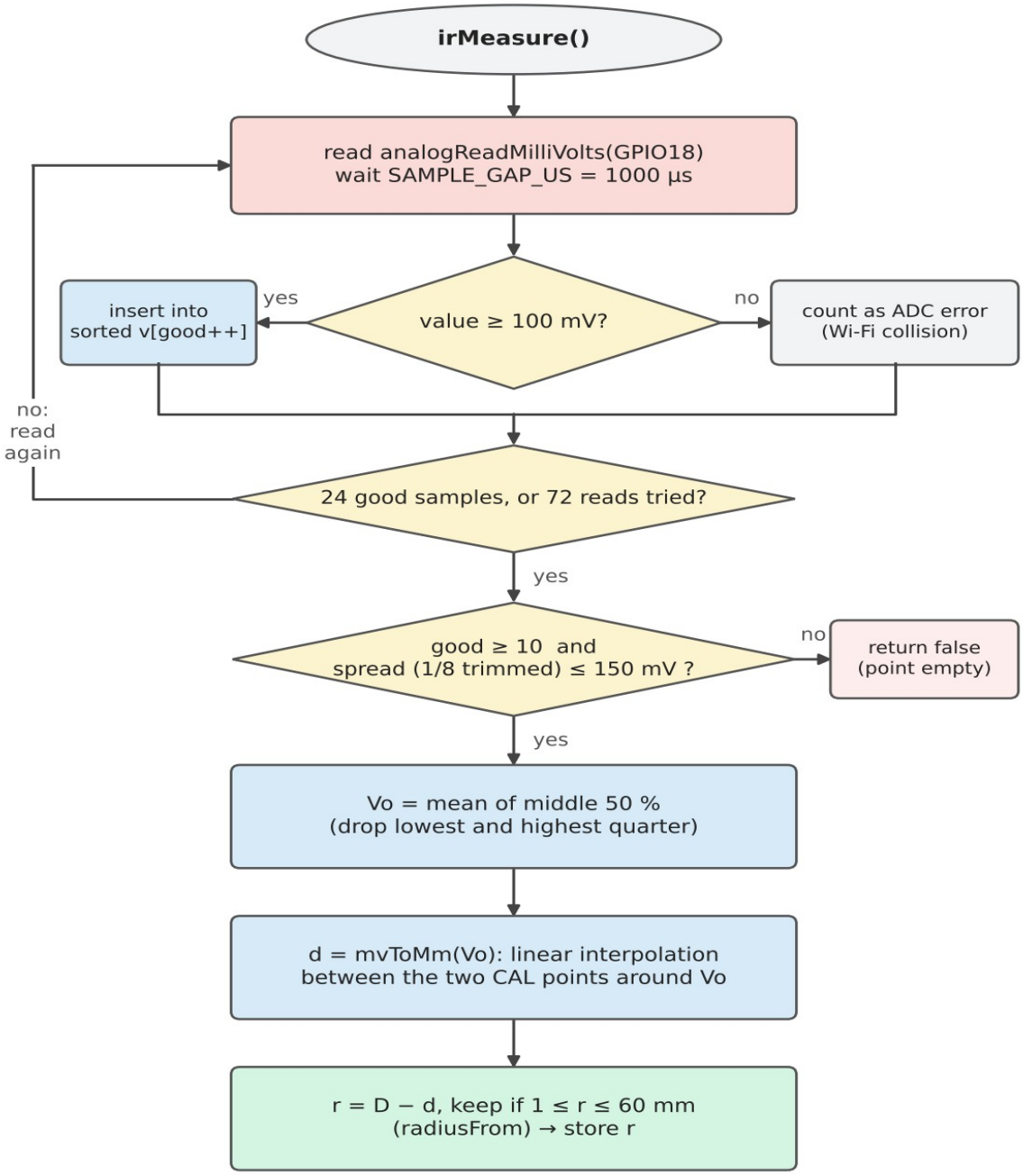


Figure 3.7. *Flowchart of the point measurement function irMeasure()*

After a layer is complete, `rejectOutliers()` compares every point with the median of its two neighbours on each side (wrapping around 360°). A point is removed if it has no valid neighbour at all, or if it differs from that median by more than 5 mm. This removes mixed edge points and single spikes without smoothing real corners.

Listing 3.3 gives the code of `irMeasure()`. Each valid sample is inserted into a sorted array, failed ADC2 conversions below `BAD_READ_MV` are only counted, the

spread between the 1/8 and 7/8 quantiles is checked, and the trimmed mean of the middle half is converted to a distance.

Listing 3.3. *Robust point measurement next to Wi-Fi, irMeasure()*

```
bool irMeasure(float& dist, float& mv) {
  int v[SAMPLES], good = 0;
  mv = 0;
  for (int t = 0; t < SAMPLES * 3 && good < SAMPLES; t++) {
    int x = analogReadMilliVolts(PIN_IR);
    adcReads++;
    if (x >= BAD_READ_MV) {
      int j = good++;
      while (j > 0 && v[j - 1] > x) { v[j] = v[j - 1]; j--; }
      v[j] = x;
    } else {
      adcBad++;
    }
    delayMicroseconds(SAMPLE_GAP_US);
  }
  if (good < MIN_GOOD_SAMPLES) return false;
  if (v[good - 1 - good / 8] - v[good / 8] > SPREAD_MAX_MV) return false;
  int q = good / 4;
  long sum = 0;
  for (int i = q; i < good - q; i++) sum += v[i];
  mv   = sum / float(good - 2 * q);
  dist = mvToMm(mv);
  return dist >= 0;
}
```

3.7. ESP32 – PC communication protocol

Figure 3.8 shows the message sequence and Table 3.6 the message formats. When a PC connects, the ESP32-S3 sends START and then every layer that is already finished, so a PC that joins late still receives the whole scan from RAM. During the scan each layer is sent as soon as it is ready. When the scan is over the ESP32-S3 sends END with the total number of points. The PC compares it with the number of lines it wrote and replies ACK or NACK. On ACK the ESP32-S3 closes the server; on NACK, or if no reply arrives within 3 s, it drops the client and waits for a new connection, after which everything is sent again from layer 0.

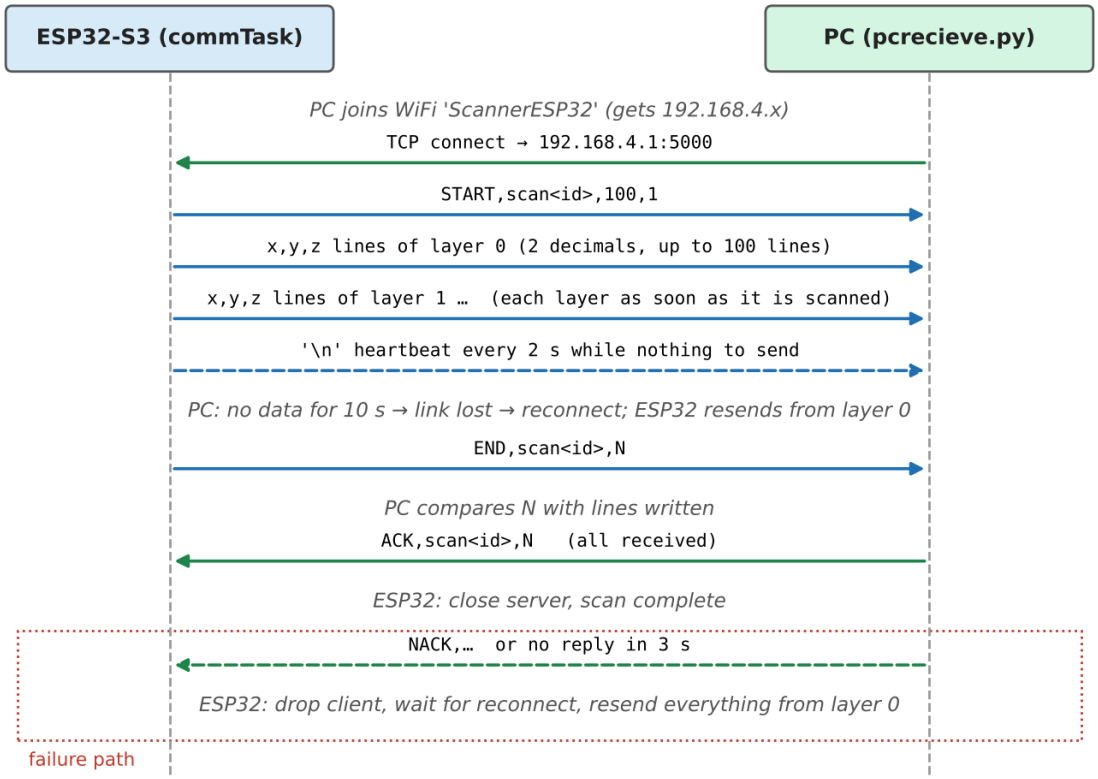


Figure 3.8. *Sequence diagram of the data exchange between the ESP32-S3 and the PC*

Table 3.6. *Messages of the scan data protocol (one ASCII line each, ending with \n)*

Direction	Message	Meaning
ESP32 → PC	START,<id>,100,1	new transfer: scan id, points per revolution, layer step (mm)
ESP32 → PC	x,y,z	one point in mm, two decimals
ESP32 → PC	(empty line)	heartbeat every 2 s when there is nothing else to send
ESP32 → PC	END,<id>,N	scan finished, N points were sent
PC → ESP32	ACK,<id>,N	N lines received and written to the file
PC → ESP32	NACK,<id>,n	count mismatch: the ESP32 will resend

Both sides watch the link. The PC treats 10 s without any data as a lost link and reconnects every 2 s; the ESP32-S3 treats a closed socket or a disappearing station as a lost link. Points are sent with two decimals (0.01 mm), which is finer than the sensor's resolution and keeps each line under 25 bytes.

Listing 3.4 shows the end of a transfer inside `commStep()`: after the scan the ESP32-S3 sends END with the number of points and waits up to `ACK_WAIT_MS` (3 s) for the answer; an ACK closes the server, while a NACK or a timeout drops the client so that the next connection receives the whole scan again.

Listing 3.4. *END line and ACK/NACK handling in commStep() (shortened)*

```
if (g_scanDone) {
  char line[64];
  int n = snprintf(line, sizeof(line), "END,%s,%ld\n", g_scanId, sentPoints);
  if (!putLine(line, n) || !flushTx()) { dropPc("loi gui"); return; }
  waitingAck  = true;
  ackDeadline = millis() + ACK_WAIT_MS;
  return;
}
...
if (waitingAck) {
  ...
    if (strncmp(ackLine, "ACK", 3) == 0) {
      finished = true;
      client.stop();
      server.end();
    } else {
      dropPc("PC bao thieu diem");
    }
  ...
  if ((int32_t)(millis() - ackDeadline) > 0) dropPc("khong nhan duoc ACK");
}
```

3.8. PC software design

3.8.1. The receiver pcrecieve.py

`pcrecieve.py` connects to 192.168.4.1:5000 and processes the stream line by line. A START line opens `scan_received.txt` for writing (overwriting the previous scan); each valid x,y,z line is appended and flushed immediately, so a partial file is available even if the transfer breaks; lines that cannot be parsed as three numbers are reported and skipped. On END the script compares the counts, replies ACK or NACK and closes the file. Options set the host, port, output file, link timeout and whether to exit after the first complete scan (`--once`).

3.8.2. The mesh builder xyztostl.py

`xyztostl.py` turns the point file into an STL model through the eight steps of Figure 3.9. Two steps are specific to this scanner. The **beam-offset correction** in `build_grid()` rebuilds each point from its measured radius r and angle θ as the true hit point ($r\cos\theta - e\sin\theta$, $r\sin\theta + e\cos\theta$), then resamples each layer back onto the regular 3.6° grid by interpolation. With `--beam-offset auto` the script searches e

from −25 to 25 mm (1 mm steps, then 0.1 mm) and keeps the value for which the middle layers best fit a square, which requires a square object such as the reference cube. The **grazing-point filter** `drop_grazing()` estimates the local surface direction from neighbouring points and removes points where the beam meets the surface at more than 60° from the normal, because these are the least reliable readings (Figure 2.5(e)).

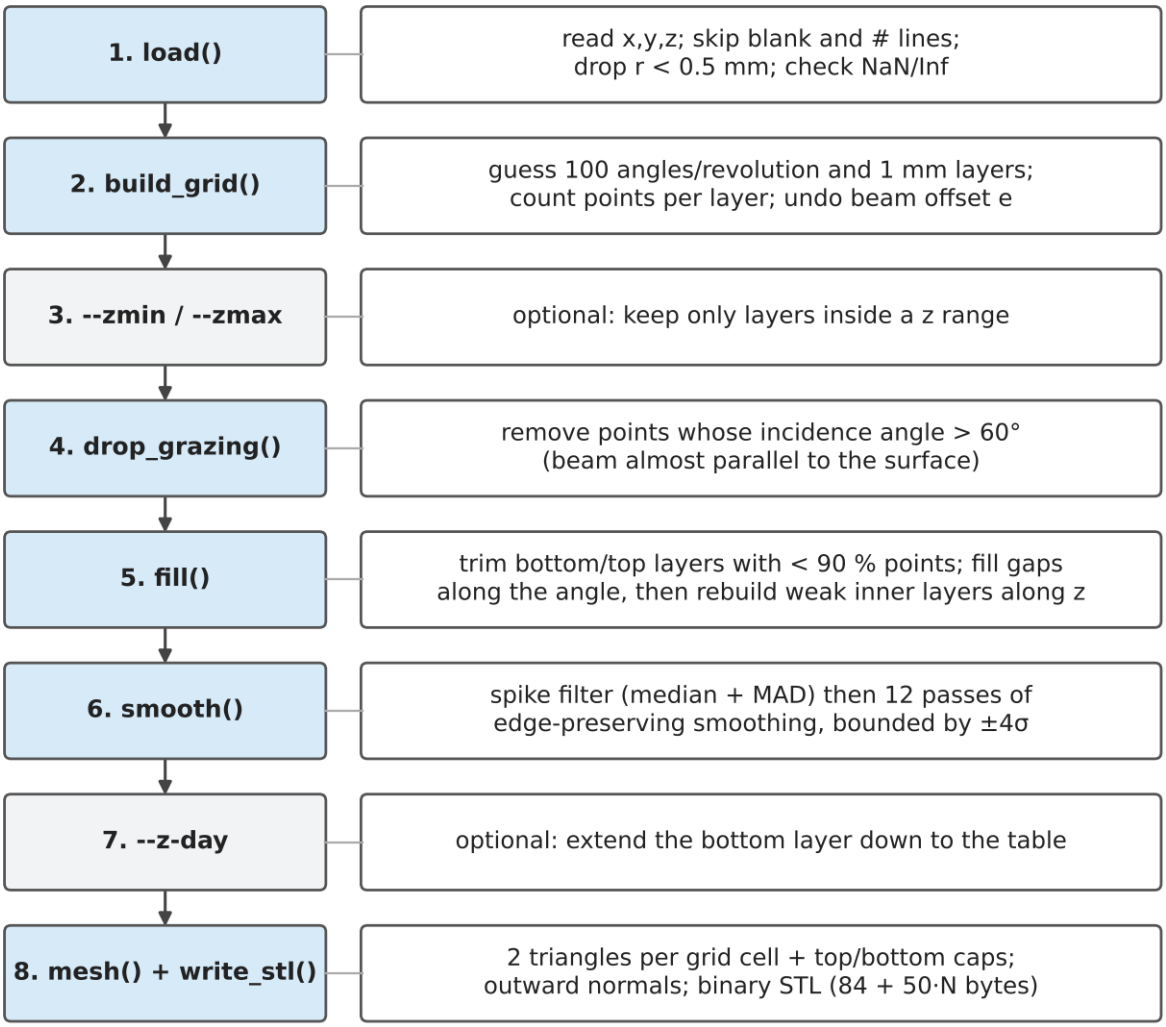


Figure 3.9. *Processing pipeline of xyztostl.py*

The number of valid points per layer is counted before the offset correction, exactly as in the firmware's Serial summary, and layers with fewer than 90 % of the expected points are trimmed from the bottom and top (the beam is partly above the object or partly below the table edge). Weak layers in the middle are rebuilt from the

layers above and below. The script prints four lines: what was cut, how much was interpolated, the model size and the output file. Table 3.7 lists the options.

Table 3.7. *Command-line options of xyztostl.py*

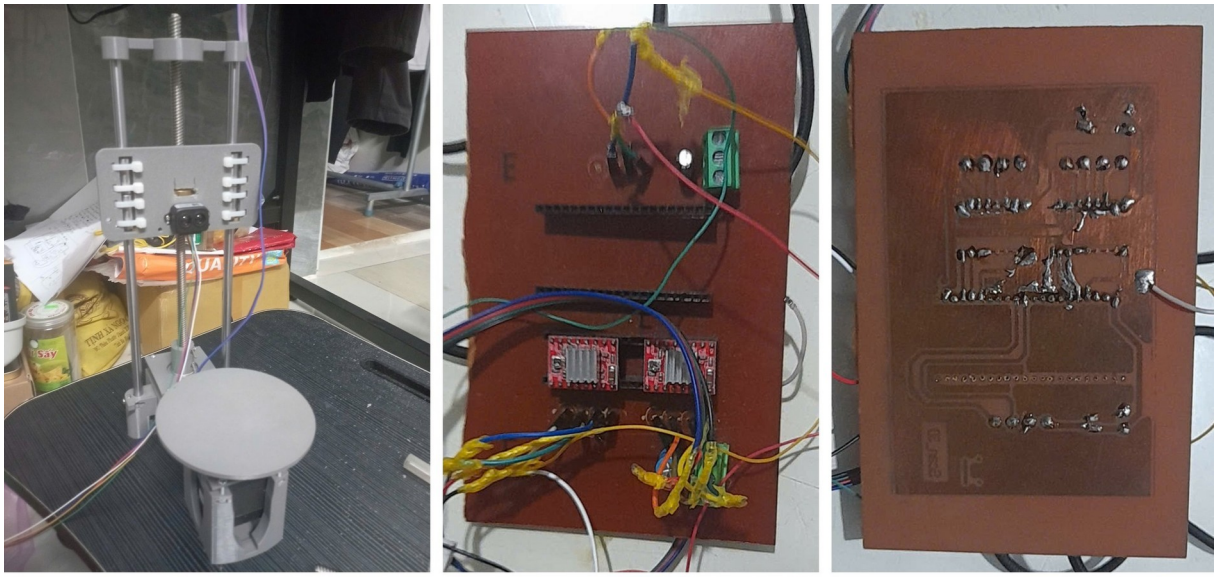
Option	Default	Meaning
input / -o	scan_received.txt / same name .stl	input point file and output STL
--beam-offset	14.1	lateral beam offset e in mm, or auto to search it on a square object
--smooth	12	number of smoothing passes; 0 disables smoothing
--min-diem	90	minimum % of points for a layer to be kept at the bottom/top
--max-incidence	60	remove points with incidence angle above this value; 0 disables
--zmin, --zmax	—	keep only layers inside this z range
--z-day	0	height of the beam above the table at z = 0; extends the model down to the table
--samples, --layer	auto	points per revolution and layer step, if automatic detection fails
--mirror	off	mirror the model if left and right are swapped

Because the scanner samples on a regular pattern (100 angles × one layer per millimetre), the mesh is built directly from the radius grid: each cell between two neighbouring layers and two neighbouring angles becomes two triangles, and the bottom and top rings are closed by fans of triangles around a centre point, which gives a closed surface. The result is written as a binary STL file: an 80-byte header, a 32-bit triangle count and one 50-byte record per triangle (normal vector and three vertices as 32-bit floats) [18]. A 57-layer scan gives 11 400 triangles and a 570 KB file. The tools are written in Python with NumPy; the optional automatic beam-offset search uses SciPy.

CHAPTER 4: IMPLEMENTATION AND EXPERIMENTS

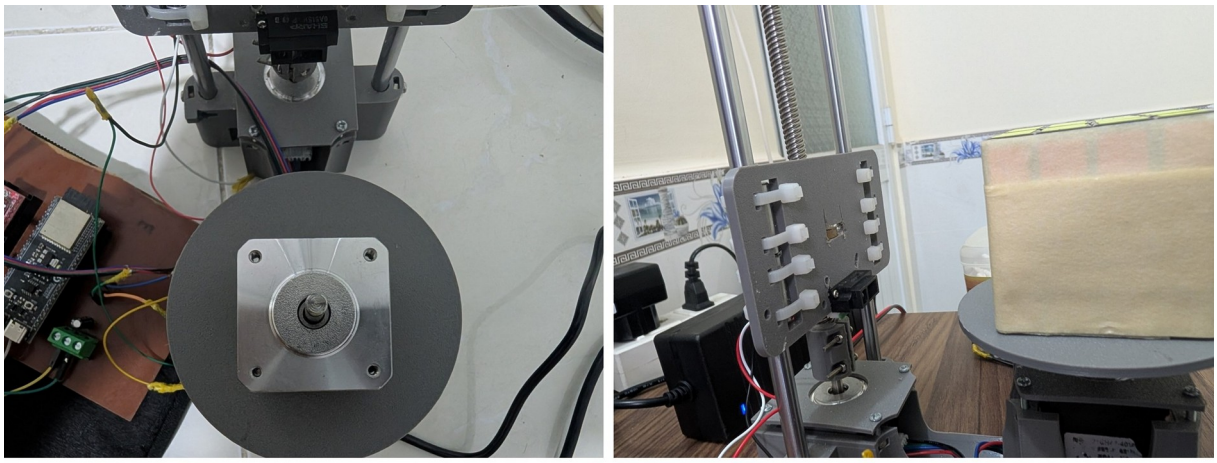
4.1. Hardware implementation

Figure 4.1 shows the printed frame and the control electronics. The control board is a single-sided PCB etched from the layout in Figure 3.4. The component side carries the female headers for the ESP32-S3-DevKitC-1, the two A4988 carriers with heat sinks, the power components and the screw terminals for the supply and the motors; the copper side holds the etched traces and the solder joints. Figure 4.2 shows the scanner fully assembled: the sensor carriage rides on two guide rods and the T8 lead screw, and the Sharp sensor faces the turntable axis horizontally. Figure 4.3 shows the reference test object: a 60 × 60 × 65 mm 4 × 4 puzzle cube, first bare and then wrapped in white A4 paper for scanning.



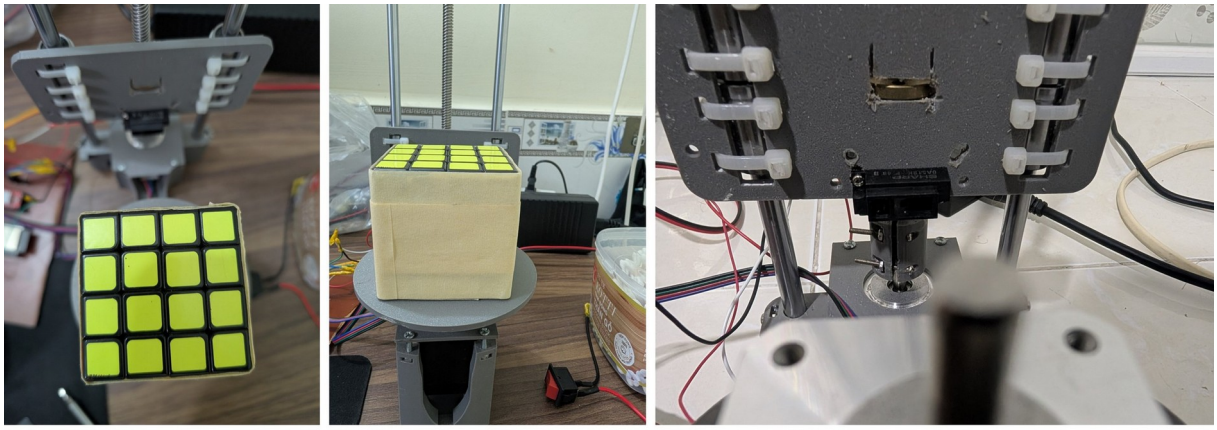
(a) printed frame (b) control board, component side (c) control board, copper side

Figure 4.1. *Hardware build: (a) printed frame, (b) component side and (c) copper side of the etched control board*



(a) Top view: ESP32-S3 board, turntable, Z carriage (b) Side view: sensor carriage on the Z axis

Figure 4.2. *The assembled scanner: (a) top view, (b) sensor carriage on the Z axis*



(a) 4×4 cube, bare (b) wrapped in A4 paper, on the table (c) sensor face seen from the table

Figure 4.3. *Reference test object and sensor position*

4.2. Unit tests of the functional blocks

4.2.1. Driver and motor test

A separate test program drove each A4988 with full steps at the timing used by the scanner, first for single revolutions of the turntable and then for Z moves with and without the ramp. The test confirmed the pin mapping and directions in Table 3.2 and showed that short, closely spaced steps made the loaded table vibrate visibly. Increasing the gap between the two steps of a point to 22 ms, together with the 30 ms settle time, made the table visibly still at the moment of measurement. In all later scans the first and last point of every layer join without a seam, which shows that no steps are lost over a revolution.

4.2.2. Sensor repeatability

The sensor test program printed one averaged reading after another while a flat white board was held at fixed distances on a ruler. Table 4.1 summarises the readings. The voltage noise is 1–7 mV (one standard deviation), but because the sensitivity falls with distance, the corresponding distance noise grows from 0.1–0.2 mm at 30–50 mm to about 1 mm at 85 mm. This is the main reason the scanner works best when the surface is close to the sensor, i.e. when the object fills most of the 60 mm radius.

Table 4.1. *Repeated readings on a flat white board (each reading is already an average of 16 samples)*

Distance (mm)	Readings	Mean Vo (mV)	SD of Vo (mV)	Sensitivity (mV/mm)	SD of distance (mm)
20	26	2191	3.4	66.8	0.05
30	29	1643	6.3	39.4	0.16
40	28	1302	5.4	27.8	0.19
50	21	1089	1.3	17.4	0.07
60	27	954	4.0	11.1	0.36
70	50	854	6.4	9.7	0.66
80	41	765	6.7	7.5	0.89
85	46	728	6.5	6.5	1.00

4.2.3. Wi-Fi and transfer test

The PC joined the ScannerESP32 network and received complete scans with `pcrecieve.py`. The ACK line in the Serial log and identical point counts on both sides confirmed every transfer. The same program was also exercised in the software-in-the-loop tests of Section 4.4, including link loss and late connection.

4.3. Calibration

4.3.1. Sensor-to-axis distance and beam offset

D was measured with a ruler from the sensor face to the turntable shaft (90 mm). The beam offset was first estimated from a photograph of the carriage (about

13.5 mm) and then determined from a scan of the cube with `--beam-offset auto`, which returned 14.1 mm. The two estimates agree within 0.6 mm, and 14.1 mm is used as the default.

4.3.2. Calibration table

The calibration table in the firmware (15 points, 30–100 mm, red curve in Figure 2.4) was built by adjusting the voltage–distance pairs until scans of the paper-wrapped cube had the correct size. Compared with the flat-board curve, it has a visible kink between 45 and 65 mm. A smoothed table was therefore considered, and its effect was simulated under five hypotheses about the real sensor, from 'the real sensor follows a smooth curve' (α = 0) to 'the real sensor follows the kinked table exactly' (α = 1).

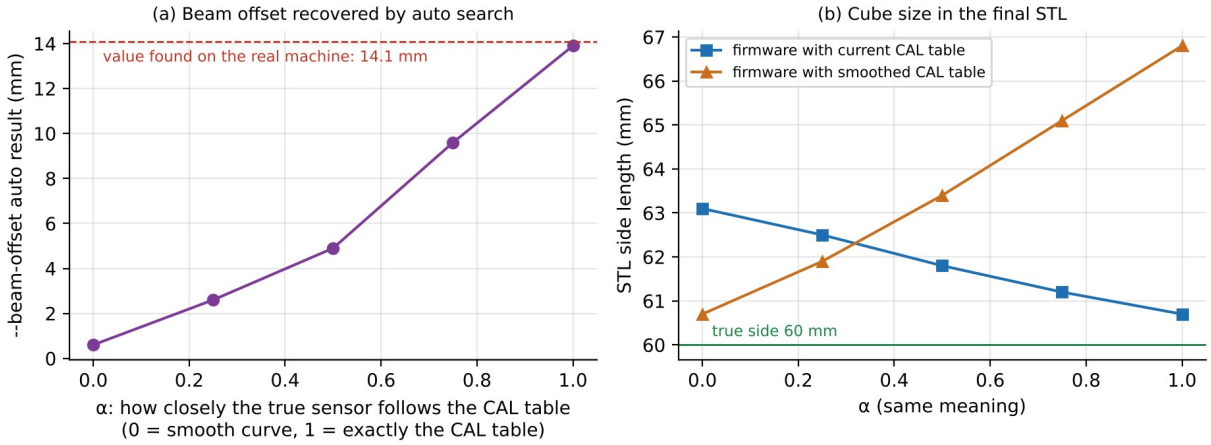


Figure 4.4. *Calibration-table simulation: (a) beam offset recovered by the auto search, (b) cube size in the final STL with the current and the smoothed table*

Figure 4.4(a) gives the decisive result. Only when the real sensor follows the current table (α = 1) does the automatic search recover a beam offset of 13.9 mm, matching the 14.1 mm found on the real machine; if the real sensor were smooth, the search would return about 0.6 mm. The kink is therefore a real property of the sensor on this object, most likely the effect of oblique incidence on the paper near the cube corners (d ≈ 50–55 mm). With the current table the cube is reconstructed at 60.7 mm; the smoothed table would make it 66.8 mm (Figure 4.4(b)). The current table was kept. The consequence is that this table is valid for objects similar to the cube; objects of other shapes or materials need their own calibration on the same surface.

4.4. Software verification by simulation

The unmodified `scan.cpp` was compiled for the PC together with small replacements for the Arduino, FreeRTOS and Wi-Fi functions, and connected to a simulated machine: a turntable with ringing, a Sharp sensor with a 16.5 ms update cycle and noise, an ADC2 that fails at a chosen rate while Wi-Fi is busy, and a 60 × 60 mm block placed 0.5 mm off-centre with a 14.1 mm beam offset. The Wi-Fi replacement opens real TCP sockets, so the real `pcrecieve.py` could connect to the simulated scanner. Each program was run at least five times without changing any line of code: 60 test runs in total, or 68 program runs, because the eight real-socket tests run scan.cpp and pcrecieve.py together. Table 4.2 summarises the results.

Table 4.2. *Software-in-the-loop test results*

Program	Scenario	Runs	Result
scan.cpp	build with all warnings (-Wall -Wextra)	1	no warnings in scan.cpp
scan.cpp	normal scan, 5 different noise seeds	5	58 layers, 5700 points, point error 0.29–0.30 mm RMS, Z home, ACK
scan.cpp	no object on the table	1	stops after 5 layers (41 s), reports 'no object', END with 0 points
scan.cpp	PC never connects / connects after the scan	2	scan completes in RAM; late PC receives all 5700 points
scan.cpp	Wi-Fi lost mid-scan / PC answers NACK once	2	full resend from layer 0, ACK on second attempt
scan.cpp	36 % ADC2 failures	1	still 100/100 points per layer, 16 % slower
scan.cpp	object taller than 150 mm	1	stops at 150 mm (151 layers, 20 min), Z returns home
scan.cpp	sensor curve differs from the CAL table	1	STL 64.7 mm, error 1.72 mm (calibration matters)
scan.cpp + pcrecieve.py	8 real-socket scenarios (late PC, restart, frozen ESP, long layer, intruder, rescan…)	8	correct point count in every case; frozen link detected after 10 s

Program	Scenario	Runs	Result
pcrecieve.py	replay of 5 simulated scans over TCP	5	5700/5700 points, file identical line by line, ACK
pcrecieve.py	malformed lines	5	bad point lines skipped; incomplete START/END lines stop the script
xyztostl.py	5 received scans	5	60.7–60.9 × 60.7–60.8 × 56 mm, surface error 0.35 mm RMS
xyztostl.py	tall object, ADC failures, wrong CAL	3	60.7 / 60.9 / 64.7 mm, as expected from the input
xyztostl.py	all options and special inputs	10	correct output; all STL files closed with positive volume
xyztostl.py	invalid input (bad lines, empty file, NaN, one layer, bad options)	10	clear 'Loi:' message, exit code 1, no broken STL

The tests found no functional error in the three programs. They also identified a few cases worth knowing: the scanner has no Z home switch, so a power cut in the middle of a scan makes the next scan start from a wrong origin; reaching the 150 mm limit ends the scan with the same 'finished' message as a normal scan; `pcrecieve.py` stops with a Python exception if a START or END line arrives without its fields, which cannot happen with the current firmware because TCP protects the content; and the printed model size is the axis-aligned bounding box, so a cube placed at 32° to the axes is reported as 79 × 79 mm.

The simulator was also used to see whether measuring more slowly improves accuracy (Figure 4.5). Doubling `SETTLE_MS` to 60 ms adds three minutes per scan and changes nothing, because the vibration is already cancelled by the step spacing. Spreading the samples over a longer window helps: 48 samples 2.5 ms apart reduce the 95th-percentile error from 0.54 mm to 0.33 mm at the cost of 2.3 times the scan time. These gains concern random noise only; they do not remove systematic errors caused by the surface or the calibration.

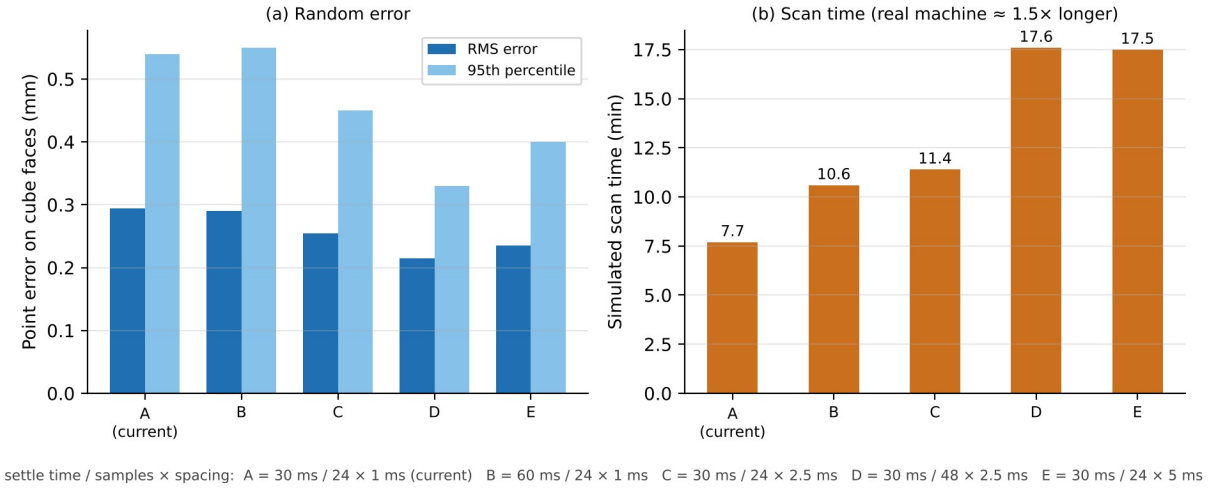


Figure 4.5. *Effect of measurement settings on random error and scan time (simulation, 60 mm cube)*

4.5. Scanning experiments

4.5.1. Effect of the beam offset

Figure 4.6 shows why the offset correction is essential. With the beam 14.1 mm off-axis, each flat face of the cube is seen at a varying angle while the table turns, and the uncorrected section becomes a pinwheel about 71 × 71 mm. The same data processed with `--beam-offset 14.1` gives a square of 60.8 × 60.9 mm. This pinwheel shape was exactly what the first real scans looked like.

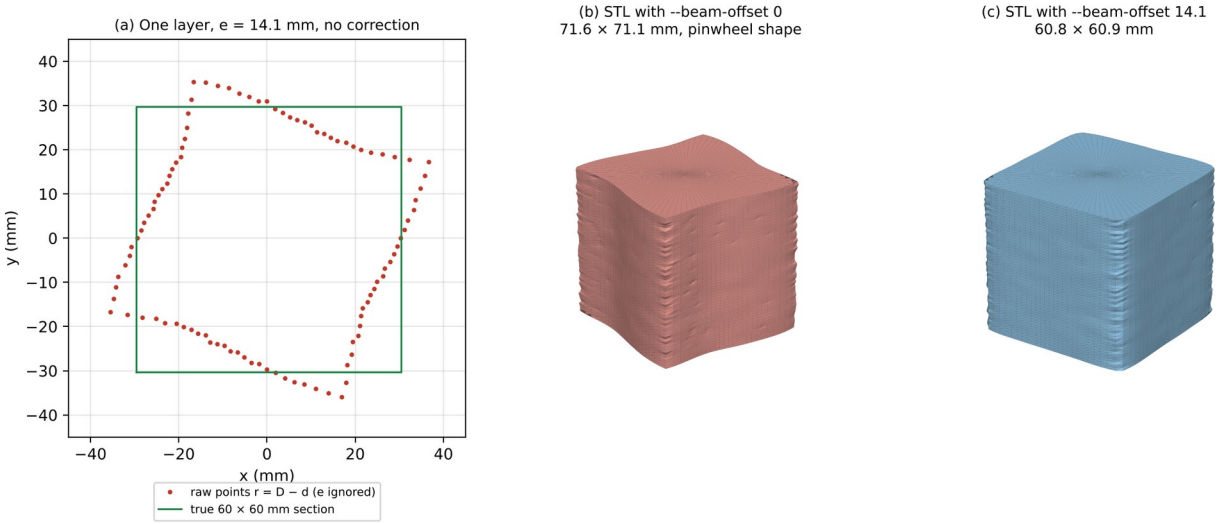


Figure 4.6. *Effect of the 14.1 mm beam offset on a simulated 60 mm cube: (a) one raw layer, (b) STL without correction, (c) STL with correction*

4.5.2. Real scan of the paper-wrapped cube

Figure 4.7 is drawn from the firmware's Serial summary of a real scan of the paper-wrapped cube, recorded in an earlier run with 0.6 mm layers; the per-layer statistics do not depend on the layer step. Every layer up to z = 55.8 mm has 100 valid points out of 100, the ADC failure rate is 0.0 % in every layer, and with 1 mm layers each layer takes about 7.9 s, i.e. about 79 ms per point. The median voltage and radius stay nearly constant along the height, which shows that the measurement is stable over the whole scan of about 8 minutes. Above about 55 mm the number of points falls quickly: the beam, which is about 10 mm above the table at z = 0, reaches the top of the 65 mm cube, consistent with its true height.

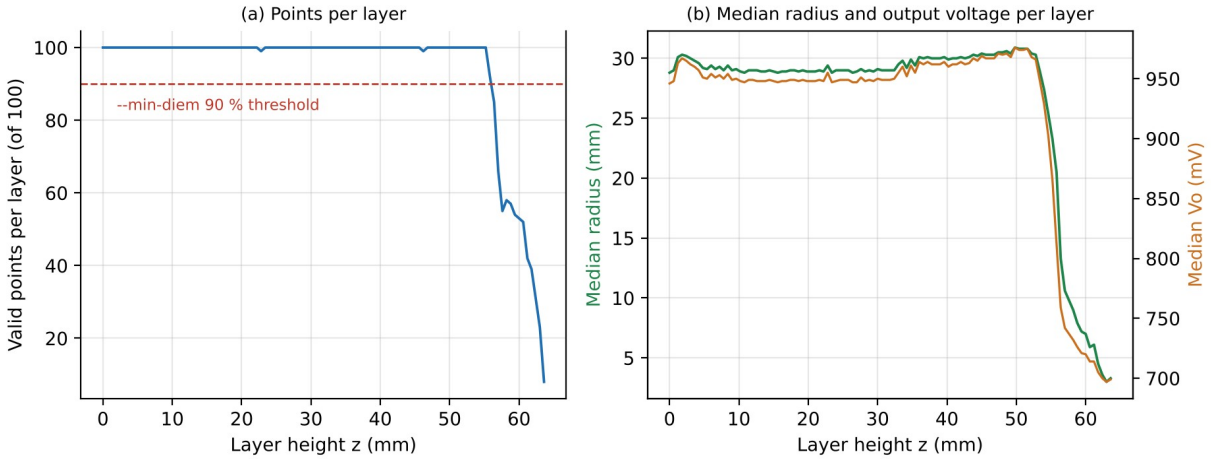


Figure 4.7. *Layer statistics from the Serial log of a real scan of the paper-wrapped cube (earlier run with 0.6 mm layers)*

Figure 4.8 compares the reference model of the cube, a simulated scan processed by the same software, the first real scan made with masking tape and an older calibration table, and the final real scan of the paper-wrapped cube made with the current table and 1 mm layers (62 layers, 12 400 triangles, 606 KB). A square fitted to each layer of the final scan has a side of 61.3 mm, 1.3 mm larger than the cube, and the faces deviate from that square by 1.2 mm on average. The corners are rounded by the width of the infrared spot and by the smoothing, so the bounding rectangle aligned with the faces is slightly larger, 62.3 × 62.5 mm. Table 4.3 summarises the dimensional results.

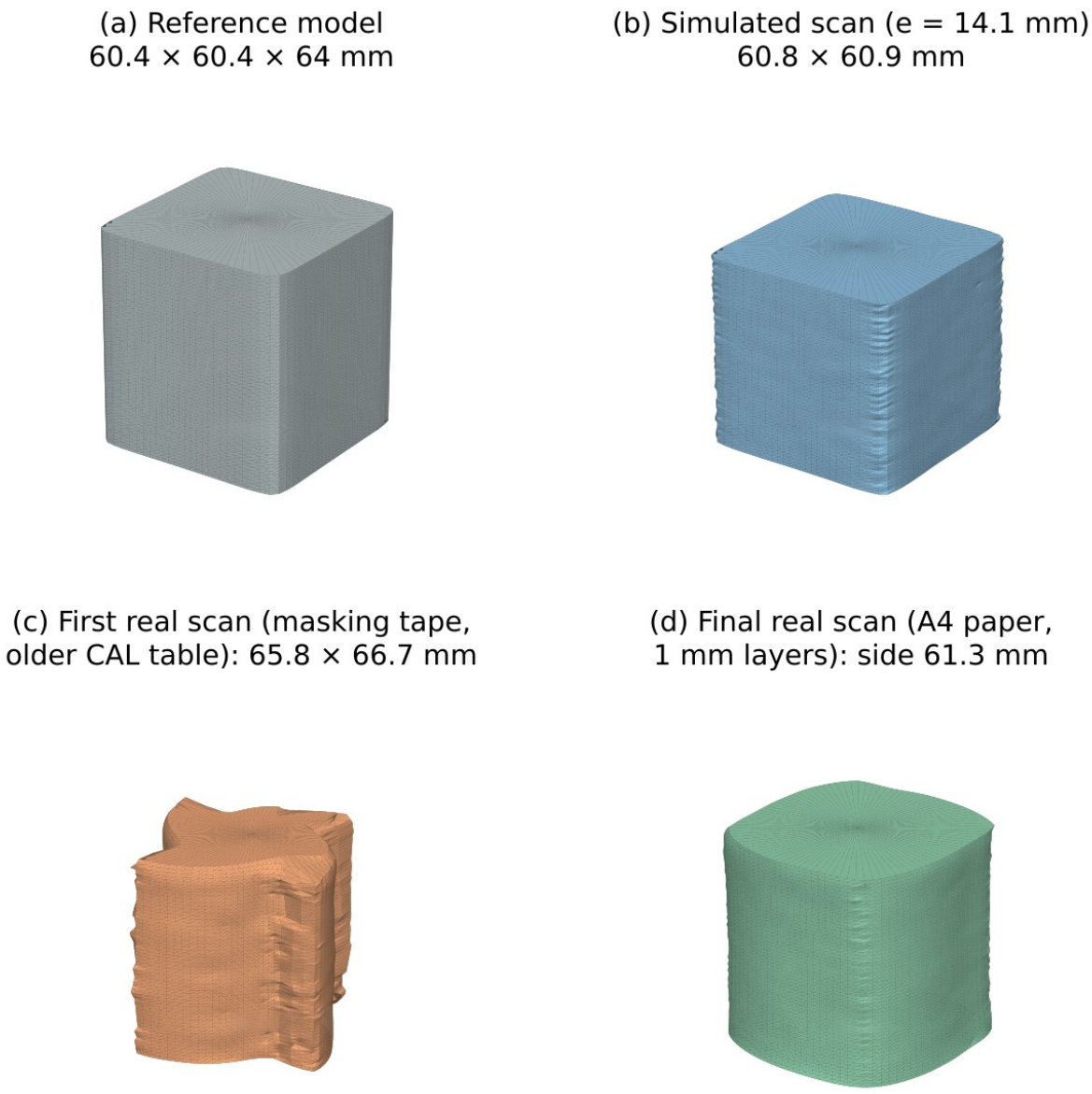


Figure 4.8. *STL models: (a) reference, (b) simulated scan, (c) first real scan with masking tape, (d) final real scan of the paper-wrapped cube*

Table 4.3. *Dimensional results for the 60 × 60 × 65 mm cube*

Case	Width × depth (mm)	Error of side	Remarks
Simulation, no beam-offset correction	71.6 × 71.1	+11.4 mm	pinwheel section
Simulation, e = 14.1 mm corrected	60.8 × 60.9	+0.8 mm	surface error 0.35 mm RMS

Case	Width × depth (mm)	Error of side	Remarks
Real, masking tape, older CAL table	65.8 × 66.7	+6.3 mm	translucent tape, faces pulled in, corners bulge
Real, A4 paper, current CAL table, 1 mm layers	62.3 × 62.5 (fitted side 61.3)	+1.3 mm	mean face deviation 1.2 mm, rounded corners, 0 % ADC errors

4.6. Problems encountered and solutions

Table 4.4. *Main problems met during development and how they were solved*

Problem	Root cause	Solution
TF-Luna readings meaningless at 30–90 mm	object inside the 0.2 m blind zone	replaced by Sharp GP2Y0A51SK0F
Readings of 0 mV in bursts	ADC2 shared with Wi-Fi	discard < 100 mV, oversample, report failure rate
Pinwheel-shaped models	beam 14 mm beside the axis	software correction --beam-offset (auto search)
Faces pulled in by 11–15 mm	translucent tape and glossy cube surface	wrap the object in matt A4 paper; recalibrate on it
Turntable vibration at measurement time	full-step ringing of the loaded table	two steps 22 ms apart + 30 ms settle
Incomplete first revolution before Z rises	early firmware started Z too soon	Z moves only after all 100 points of a layer
Ragged top and bottom rings	beam partly over the object edge	trim layers with < 90 % points; close with a cap
Spikes at cube corners	grazing incidence and mixed edge points	neighbour filter in firmware, incidence filter on PC
Model floats 10 mm above the table	no Z home switch; beam 10 mm above table at z = 0	--z-day option extends the base

4.7. Evaluation against the objectives

All functional objectives were met: the machine scans automatically from power-on to return home, measures 100 points per layer, sends every layer reliably over Wi-Fi while keeping data only in RAM, and produces closed STL models. The ADC2 restriction was handled without changing the PCB: in the real scan no conversion failed, and in simulation even a 36 % failure rate left every layer complete. Simulation shows that the processing chain itself is accurate to better than 1 mm on a 60 mm cube. On the real machine the remaining error of about 1.2–1.3 mm comes from the optical behaviour of the infrared sensor on the object surface and from calibration, not from the motion system or the software. The ±1–2 mm accuracy target is therefore met: about 0.8 mm in simulation, and a side error of 1.3 mm with a mean face deviation of 1.2 mm on the real cube. The real result must be read with one caveat. The calibration table was tuned on the same paper-wrapped cube that was then used to measure the error (Section 4.3.2), so the real-scan figures show that the scanner can reproduce this object, not that it reaches the same accuracy on any object. An independent check would require scanning a second object of the same matt material that was not used for calibration.

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1. Results achieved

- A complete low-cost 3D scanner was built with an ESP32-S3, two NEMA17/A4988 axes and a Sharp GP2Y0A51SK0F infrared triangulation sensor, replacing the TF-Luna whose minimum range made it unusable.
- The firmware uses two pinned FreeRTOS tasks, measures robustly on ADC2 next to Wi-Fi, filters outliers on the device, stops automatically and keeps all data in RAM.
- A framed TCP protocol with point counts, ACK/NACK, heartbeat and full resend makes the transfer verifiable and recoverable.
- The PC software corrects the 14.1 mm beam offset, removes unreliable grazing points, fills gaps and outputs closed binary STL files.
- The three programs passed 60 software-in-the-loop tests (68 program runs); a 60 mm cube is reconstructed to 60.7–60.9 mm in simulation, and the real paper-wrapped cube is scanned at about 7.9 s per layer with no ADC failures and a side error of 1.3 mm, within the ±1–2 mm target.

5.2. Limitations

- The infrared sensor depends on surface finish and incidence angle; glossy, dark or translucent objects must be covered with matt material, and the calibration table is only valid for similar surfaces.
- The calibration table was tuned on the same cube that was used to measure the accuracy, so the real-scan result is not an independent validation; other objects need their own check.
- The distance noise grows to about 1 mm near 90 mm from the sensor, so small objects far from the sensor are scanned less accurately.
- There is no Z home switch: z = 0 is wherever the carriage is at power-on.
- The horizontal beam cannot see the top surface or deep concave regions; the top is closed by a flat cap.

- Scanning is slow (about 7.9 s per 1 mm layer, about 8 minutes for the 65 mm cube and 20 minutes for the full 150 mm height), because one point is measured at a time.
- The scanner records geometry only, without colour.

5.3. Future work

5.3.1. Hardware and firmware improvements

- Add a limit switch at the bottom of the Z axis and home the carriage before every scan, so that heights are absolute and a power cut cannot shift the origin.
- Move the sensor to an ADC1 pin, which is not shared with Wi-Fi, or use an external 16-bit ADC such as the ADS1115 over I²C to remove ADC2 conflicts and gain resolution at long distance.
- Spread the 24–48 samples of each point over a longer window, as suggested by Figure 4.5, when accuracy matters more than time.
- Consider a laser triangulation sensor with a smaller spot and a range matched to 30–90 mm, which is less sensitive to surface finish.

5.3.2. AI-assisted automatic calibration

Today the calibration table is adjusted by hand for each surface. A learned model could do this automatically. The idea is to scan a small set of reference objects with known geometry (a cylinder, a cube and a cone) on the same surface material, and train a small regression model, for example a neural network or gradient-boosted trees, that predicts the true distance from the raw voltage, the incidence angle estimated from neighbouring points, and the height. Farias et al. showed that a neural network can replace manual calibration of nonlinear infrared proximity sensors [19], and the cubic polynomial used in the ElectroNoobs scanner [2] gives a simple baseline to compare against. The model could also estimate the beam offset and D jointly, instead of searching only e on a square object. Because the model is small, it can run on the PC or even on the ESP32-S3, and the user would calibrate a new material simply by scanning the reference objects once.

5.3.3. Scanning the top surface

A single rotation axis cannot capture every side of an object. Two extensions are possible: a second Sharp sensor mounted above the table and pointing downwards, moved radially by a small linear axis, or a tilting sensor carriage that scans the object again at 45°. Both produce additional point sets that must be registered with the side scan; because all positions come from the same stepper motors, the registration is a known rigid transform rather than a search.

5.3.4. Colour output

Colour can be captured by adding an RGB colour sensor next to the distance sensor, or a small camera module that photographs the object at each angle. Each point then receives a colour, and the model is exported as a coloured PLY or OBJ file instead of STL. Methods developed for RGB-D cameras, such as colour-map optimisation and coloured point-cloud registration, show how to blend colour from several views consistently [20].

5.3.5. AI point-cloud completion and repair

Point-cloud completion networks such as PCN can predict the missing top surface and occluded regions from an incomplete scan [21]. Integrating such a model after `xyztostl.py` would fill holes more plausibly than linear interpolation and could repair damaged or deformed objects before printing.

5.4. Conclusion

The project shows that a useful desktop 3D scanner can be built with a single analog infrared sensor and an ESP32-S3, provided that the embedded system is designed around the sensor's real behaviour: oversampled ADC readings that survive Wi-Fi interference, motion timed to cancel vibration, a verifiable wireless protocol, and PC processing that corrects the geometry of the sensor mounting. The remaining limits are optical rather than electronic, which points clearly to the next steps: better calibration, possibly learned automatically, and additional views and sensors.

REFERENCES

[1] Media Computing Group, RWTH Aachen University, "FabScan 100 – an open-source 3D laser scanner," project page. Available: https://hci.rwth-aachen.de/fabscan100

[2] ElectroNoobs, "Homemade 3D scanner with Arduino and IR sensor," tutorial page. Available: https://electronoobs.com/tutorial/homemade-3d-scanner-with-arduino-and-ir-sensor

[3] F. Blais, "Review of 20 years of range sensor development," *Journal of Electronic Imaging*, vol. 13, no. 1, pp. 231–240, 2004, doi: 10.1117/1.1631921. Available: https://www.cs.columbia.edu/~allen/PHOTOPAPERS/Blais_JEI04_Review20YearsRangeSensor.pdf

[4] Sharp Corporation, "GP2Y0A51SK0F – Distance Measuring Sensor Unit, Measuring distance: 2 to 15 cm, Analog output type," datasheet, Sheet No. OP13007EN. Available: https://global.sharp/products/device/lineup/data/pdf/datasheet/gp2y0a51sk_e.pdf

[5] Espressif Systems, "ESP32-S3 Series Datasheet," version 2.2. Available: https://documentation.espressif.com/esp32-s3_datasheet_en.pdf

[6] Allegro MicroSystems, "A4988 – DMOS Microstepping Driver with Translator and Overcurrent Protection," datasheet. Available: https://www.allegromicro.com/-/media/files/datasheets/a4988-datasheet.pdf

[7] M. Daneshmand, A. Helmi, E. Avots, F. Noroozi, F. Alisinanoglu, H. S. Arslan, J. Gorbova, R. E. Haamer, C. Ozcinar and G. Anbarjafari, "3D Scanning: A Comprehensive Survey," arXiv:1801.08863, 2018. Available: https://arxiv.org/abs/1801.08863

[8] Benewake (Beijing) Co., Ltd., "TF-Luna LiDAR Module – Product User Manual," rev. 2024. Available: https://en.benewake.com/uploadfiles/2024/04/20240426135946148.pdf

[9] Sharp Corporation, "GP2Y0A Series / GP2Y0D Series Application Note," Sheet No. OP14030EN (copy hosted by Linköping University). Available: https://www.da.isy.liu.se/vanheden/pdf/sharp-GP2Y0A-series-appnote.pdf

[10] Hamamatsu Photonics, "PSD (position sensitive detector) – technical information," KPSD9001E. Available: https://www.hamamatsu.com/resources/pdf/ssd/psd_kpsd9001e.pdf

[11] Pololu Corporation, "Sharp GP2Y0A51SK0F Analog Distance Sensor 2–15 cm," product page #2450. Available: https://www.pololu.com/product/2450

[12] G. Benet, F. Blanes, J. E. Simó and P. Pérez, "Using infrared sensors for distance measurement in mobile robots," *Robotics and Autonomous Systems*, vol. 40, no. 4, 2002, doi: 10.1016/S0921-8890(02)00271-3. Available: https://panorama.upv.es/en/ipublic/item/10198036

[13] N. Vukašinović, J. Možina and J. Duhovnik, "Correlation between incident angle, measurement distance, object colour and the number of acquired points at CNC laser scanning," *Strojniški vestnik – Journal of Mechanical Engineering*, vol. 58, no. 1, pp. 23–28, 2012, doi: 10.5545/sv-jme.2011.053. Available: https://www.sv-jme.eu/?p=2929

[14] Espressif Systems, "ADC – Arduino ESP32 API documentation" (analogReadMilliVolts, analogSetPinAttenuation). Available: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html

[15] Espressif Systems, "FreeRTOS (IDF) – ESP32-S3," ESP-IDF Programming Guide. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/freertos_idf.html

[16] Espressif Systems, "Analog to Digital Converter (ADC) Oneshot Mode Driver – ESP32-S3," ESP-IDF Programming Guide. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/peripherals/adc/adc_oneshot.html

[17] Espressif Systems, "Wi-Fi API – Arduino ESP32 documentation" (softAP, server examples). Available: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/wifi.html

[18] Library of Congress, "STL (STereoLithography) File Format, Binary," Sustainability of Digital Formats, FDD ID fdd000505. Available: https://www.loc.gov/preservation/digital/formats/fdd/fdd000505.shtml

[19] G. Farias, E. Fabregas, E. Peralta, H. Vargas, G. Hermosilla, G. Garcia and S. Dormido, "A neural network approach for building an obstacle detection model by fusion of proximity sensors data," *Sensors*, vol. 18, no. 3, art. 683, 2018, doi: 10.3390/s18030683. Available: https://www.mdpi.com/1424-8220/18/3/683

[20] Q.-Y. Zhou and V. Koltun, "Color map optimization for 3D reconstruction with consumer depth cameras," *ACM Transactions on Graphics (SIGGRAPH)*, 2014. Available: https://www.cs.princeton.edu/courses/archive/fall16/cos526/papers/zhou14.pdf

[21] W. Yuan, T. Khot, D. Held, C. Mertz and M. Hebert, "PCN: Point completion network," in *Proc. International Conference on 3D Vision (3DV)*, 2018, pp. 728–737. Available: https://arxiv.org/abs/1808.00671